

Lab 1: Comparison of Virtual Machines (VMs) and Containers

1. Objective

The primary goals of this experiment are:

- To understand the conceptual and practical differences between Virtual Machines (VMs) and Containers.
 - To install and configure an Ubuntu-based Nginx web server using both VirtualBox/Vagrant and Docker inside WSL.
 - To compare resource utilization, performance, and operational characteristics of both environments.
-

2. Hardware and Software Requirements

Hardware

- 64-bit system with virtualization enabled in BIOS
- Minimum 8 GB RAM (4 GB acceptable)
- Internet connection

Software

- Oracle VirtualBox
 - Vagrant
 - Windows Subsystem for Linux (WSL 2) with Ubuntu
 - Docker Engine (docker.io)
-

3. Theory

Feature	Virtual Machine (VM)	Container
Virtualization Level	Emulates complete hardware and kernel	Virtualizes at OS level, shares host kernel
Isolation	Strong (Full OS isolation)	Moderate (Process-level isolation)
Resource Usage	Higher (Dedicated RAM/CPU for OS)	Lightweight and efficient
Startup Time	Slower (Seconds/Minutes)	Fast (Milliseconds/Seconds)

4. Experiment Setup – Part A: Virtual Machine

Step 1: Initialization and Deployment

Using Vagrant, an Ubuntu VM was initialized and started.

```
vagrant init ubuntu/jammy64
vagrant up
```

```
PowerShell 7 (x64)
'vagrantup.com' for more information on using Vagrant.
PS C:\Users\MP\vm-lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
=> default: Box 'ubuntu/jammy64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
=> default: Loading metadata for box 'ubuntu/jammy64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/ubuntu/jammy64
=> default: Adding box 'ubuntu/jammy64' (v20241002.0.0) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/ubuntu/boxes/jammy64/versions/20241002.0.0/providers/virtualbox/unknown/vagrant.box
    default:
=> default: Successfully added box 'ubuntu/jammy64' (v20241002.0.0) for 'virtualbox'!
=> default: Importing base box 'ubuntu/jammy64'...
    default: URL: https://vagrantcloud.com/api/v2/vagrant/ubuntu/jammy64
=> default: Matching MAC address for NAT networking...
=> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
=> default: Setting the name of the VM: vm-lab_default_1769365194678_10977
=> default: Clearing any previously set network interfaces...
=> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
=> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
=> default: Running 'pre-boot' VM customizations...
=> default: Booting VM...
=> default: Waiting for machine to boot. This may take a few minutes...
    default: SSH address: 127.0.0.1:2222
    default: SSH username: vagrant
    default: SSH auth method: private key
    default: Warning: Connection reset. Retrying...
    default: Warning: Connection aborted. Retrying...
    default:
    default: Vagrant insecure key detected. Vagrant will automatically replace
    default: this with a newly generated keypair for better security.
    default:
    default: Inserting generated public key within guest...
    default: Removing insecure key from the guest if it's present...
    default: Key inserted! Disconnecting and reconnecting using new SSH key...
=> default: Machine booted and ready!
=> default: Checking for guest additions in VM...
    default: The guest additions on this VM do not match the installed version of
    default: VirtualBox! In most cases this is fine, but in rare cases it can
    default: prevent things such as shared folders from working properly. If you see
```

Observation:

The system downloads the Ubuntu Jammy base box and configures the VirtualBox provider. Port forwarding (2222 → 22) is established.

Step 2: Accessing the VM (SSH)

```
vagrant ssh
```

```
PS C:\Users\MP\vm-lab> vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-164-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun Jan 25 18:22:35 UTC 2026

System load:          0.69
Usage of /:            4.0% of 38.7GB
Memory usage:         20%
Swap usage:           0%
Processes:            108
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::28:41ff:fee4:55a4

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vagrant@ubuntu-jammy:~$ |
```

Observation:

Successful login to Ubuntu 22.04.5 LTS environment.

Step 3: Installing Nginx

```
sudo apt update
sudo apt install -y nginx
```

```
vagrant@ubuntu-jammy: ~
Reading state information... Done
9 packages can be upgraded. Run 'apt list --upgradable' to see them.
vagrant@ubuntu-jammy:~$ sudo apt install -y nginx
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libdeflate0 libfontconfig1 libgd3 libjpeg8 libjpeg-turbo8 libjpeg9 libnginx-mod-http-geoip2
  libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libnginx-mod-stream-geoip2 libtiff5 libwebp7 libxpm4
  nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libdeflate0 libfontconfig1 libgd3 libjpeg8 libjpeg-turbo8 libjpeg9 libnginx-mod-http-geoip2
  libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libnginx-mod-stream-geoip2 libtiff5 libwebp7 libxpm4
  nginx nginx-common nginx-core
0 upgraded, 20 newly installed, 0 to remove and 9 not upgraded.
Need to get 2694 kB of archives.
After this operation, 8366 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-dejavu-core all 2.37-2build1 [1441 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fontconfig-config all 2.13.1-4.2ubuntu5 [29.1 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 libdeflate0 amd64 1.18-2 [70.9 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/main amd64 libfontconfig1 amd64 2.13.1-4.2ubuntu5 [131 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjpeg-turbo8 amd64 2.1.2-0ubuntu1 [134 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjpeg8 amd64 8c-2ubuntu10 [2264 B]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libjpeg9 amd64 2.1-3.1ubuntu0.22.04.1 [29.2 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libwebp7 amd64 1.2.2-2ubuntu0.22.04.2 [206 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtiff5 amd64 4.3.0-6ubuntu0.12 [185 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libxpm4 amd64 1:3.5.12-1ubuntu0.22.04.2 [36.7 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgd3 amd64 2.3.0-2ubuntu2.3 [129 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 nginx-common all 1.18.0-6ubuntu14.7 [40.1 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-http-geoip2 amd64 1.18.0-6ubuntu14.7 [12.0 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-http-image-filter amd64 1.18.0-6ubuntu14.7 [15.5 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-http-xslt-filter amd64 1.18.0-6ubuntu14.7 [13.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-mail amd64 1.18.0-6ubuntu14.7 [45.8 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-stream amd64 1.18.0-6ubuntu14.7 [73.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnginx-mod-stream-geoip2 amd64 1.18.0-6ubuntu14.7 [10.1 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 nginx-core amd64 1.18.0-6ubuntu14.7 [483 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 nginx amd64 1.18.0-6ubuntu14.7 [3878 B]
Fetched 2694 kB in 4s (688 kB/s)
Preconfiguring packages ...
```

Observation:

The apt package manager installs required dependencies. This process is slower compared to Docker because a full OS environment is involved.

```
vagrant@ubuntu-jammy: ~
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
Caching processes...
Caching linux images...
Caching kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
vagrant@ubuntu-jammy:~$ sudo systemctl start nginx
vagrant@ubuntu-jammy:~$ curl localhost
DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
body {
```

Observation:

Installation completes successfully with triggers processed for man-db and ufw.

Step 4: Verification Inside VM

```
curl localhost
```

```
vagrant@ubuntu-jammy: ~$ sudo systemctl start nginx
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
vagrant@ubuntu-jammy:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">http://nginx.org/</a>.<br>
Commercial support is available at
<a href="http://nginx.com/">http://nginx.com/</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@ubuntu-jammy:~$
```

Observation:

The command returns the full HTML source of the “Welcome to nginx!” page.

5. Experiment Setup – Part B: Containers (Docker)

Step 1: Running the Container

```
docker run -dp 8080:80 --name nginx-container nginx
```

Observation:

Docker pulls image layers and starts the container almost instantly.

Step 2: Verification

```
curl localhost:8080
```

```

harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab
API version: 1.51 (downgraded from 1.53)
Go version: go1.25.6
Git commit: 0b9d198
Built: Mon Jan 26 19:27:07 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Desktop 4.46.0 (204649)
Engine:
  Version: 28.4.0
  API version: 1.51 (minimum version 1.24)
  Go version: go1.24.7
  Git commit: 249d679
  Built: Wed Sep 3 20:57:37 2025
  OS/Arch: linux/amd64
  Experimental: false
containerd:
  Version: 1.7.27
  GitCommit: 05044ec0a9a75232cad458027ca83437aae3f4da
runc:
  Version: 1.2.5
  GitCommit: v1.2.5-0-g59923ef
docker-init:
  Version: 0.19.0
  GitCommit: de40ad0
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker pull nginx
docker run -d -p 8080:80 nginx
Using default tag: latest
latest: Pulling from library/nginx
700146c8ad64: Pull complete
10b68cfefee1: Pull complete
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
57f0dd1befe2: Pull complete
119d43eec815: Pull complete
500799c30424: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
f76031f996a9ed147038793916dcb165cd26c6123d83401256617d299b8e007c
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$

```

```

Select harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab
containerd:
  Version: 1.7.27
  GitCommit: 05044ec0a9a75232cad458027ca83437aae3f4da
runc:
  Version: 1.2.5
  GitCommit: v1.2.5-0-g59923ef
docker-init:
  Version: 0.19.0
  GitCommit: de40ad0
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker pull nginx
docker run -d -p 8080:80 nginx
Using default tag: latest
latest: Pulling from library/nginx
700146c8ad64: Pull complete
10b68cfefee1: Pull complete
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
57f0dd1befe2: Pull complete
119d43eec815: Pull complete
500799c30424: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
f76031f996a9ed147038793916dcb165cd26c6123d83401256617d299b8e007c
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker run -d -p 8080:80 nginx
5ad3f18b1bc470c2b92a0338a634c30bef7cfc85d6125225bdd42f2ad3274534
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint gallant_mcnulty (0d1724c706b90bd2e44692ad483f82b56514c699e608e23e526d407224aec37d): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
f76031f996a9   nginx    "/docker-entrypoint..." 4 minutes ago  Up 4 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  bold_joliot
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ ^[[200~docker stop f76031f996a9
docker rm f76031f996a9
^[[201~docker: command not found
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker rm f76031f996a9
Error response from daemon: cannot remove container "f76031f996a9": container is running; stop the container before removing or force remove
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ ~
-bash: /home/harshita24: Is a directory
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker rmi nginx
Error response from daemon: conflict: unable to delete nginx:latest (must be forced) - container f76031f996a9 is using its referenced image c881927c4077
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$

```

Observation:

The Nginx “Welcome” page is successfully displayed on port 8080.

6. Resource Utilization & Comparison

A. Boot Time Analysis

```
systemd-analyze
```

Observation (VM):
The VM took 36.819 seconds to finish startup (6.9s kernel + 29.8s userspace).

Observation (Container):
The container started in less than 1 second.

B. Process Overhead & Isolation

```
htop
```

Observation (VM):
Multiple background services such as systemd, snapd, rsyslogd, polkitd, and sshd are running.

```
docker stats
```

Observation (Container):
The container runs only the Nginx process and minimal dependencies.

C. Memory Usage

```
free -h
```

Observation (VM):
957 Mi total allocated, with ~196 Mi used by OS services.

Observation (Container):
Container consumes approximately 13.22 MiB of RAM.

Comparison Summary Table

Parameter	Virtual Machine (VM)	Container (Docker)
Boot Time	~36.8 seconds	< 1 second
RAM Usage	~196 MiB	~13.22 MiB
Background Processes	High	Low
Disk Usage	Larger	Smaller

7. Conclusion

This experiment demonstrates that Containers are significantly more lightweight than Virtual Machines.

- **Speed:** Docker containers start instantly, while VMs require ~37 seconds to boot.
- **Efficiency:** VMs consume ~200MB RAM for OS overhead, whereas containers use ~13MB.
- **Complexity:** VMs run a full operating system stack, creating additional overhead.

Final Conclusion:

Containers are ideal for microservices and rapid scaling, while Virtual Machines are better suited for complete OS isolation and hardware-level simulation.

Lab 2: Docker Installation, Configuration, and Running Images

1. Objective

The objectives of this experiment are:

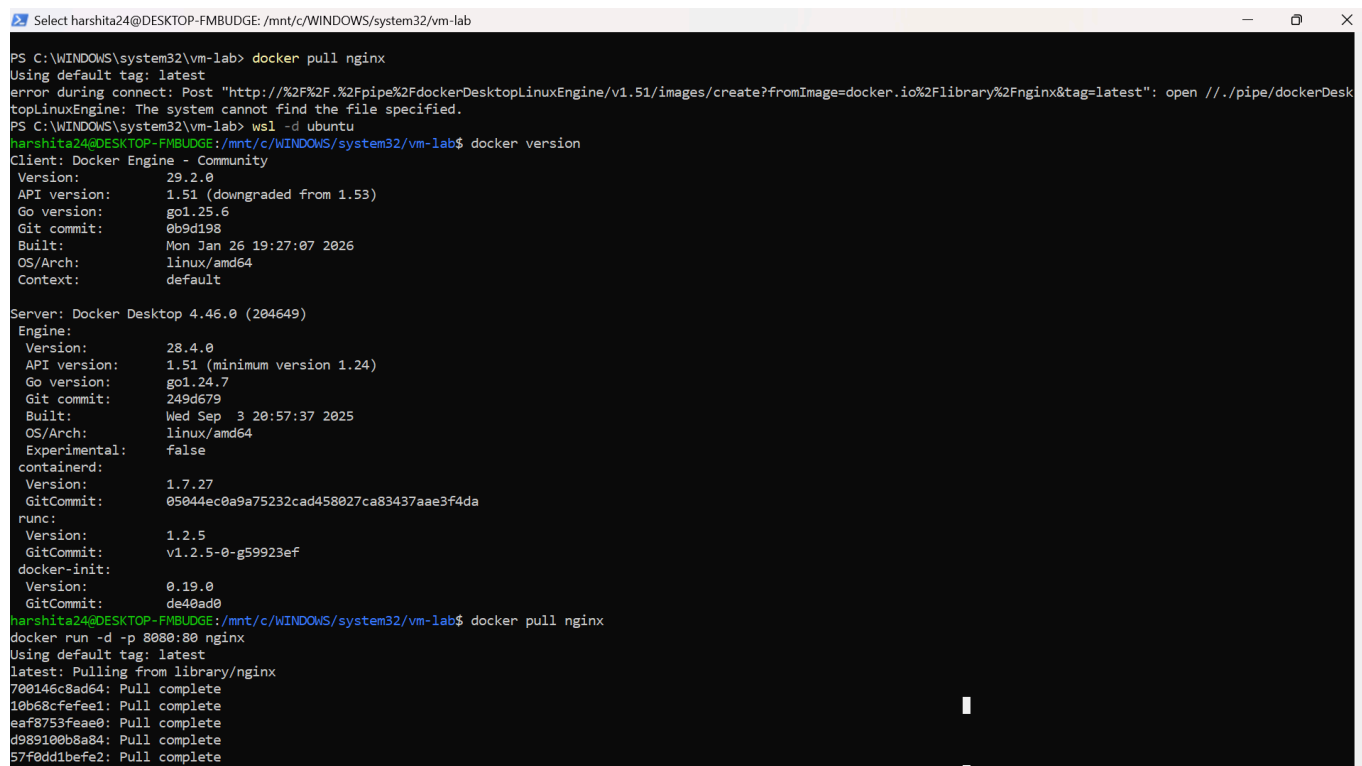
- To pull Docker images from Docker Hub
 - To run containers using Docker
 - To manage container lifecycle (start, stop, remove)
 - To remove Docker images
-

2. Procedure

Step 1: Pull Docker Image

The nginx image was pulled from Docker Hub using the following command:

```
docker pull nginx
```



```
Select harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab
PS C:\WINDOWS\system32\vm-lab> docker pull nginx
Using default tag: latest
error during connect: Post "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.51/images/create?fromImage=docker.io%2Flibrary%2Fnginx&tag=latest": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\WINDOWS\system32\vm-lab> wsl -d ubuntu
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker version
Client: Docker Engine - Community
 Version:           29.2.0
 API version:       1.51 (downgraded from 1.53)
 Go version:        go1.25.6
 Git commit:        0b9d198
 Built:             Mon Jan 26 19:27:07 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Desktop 4.46.0 (204649)
 Engine:
  Version:          28.4.0
  API version:       1.51 (minimum version 1.24)
  Go version:        go1.24.7
  Git commit:        249d679
  Built:             Wed Sep  3 20:57:37 2025
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:          1.7.27
  GitCommit:        05044ec0a9a75232cad458027ca83437aae3f4da
 runc:
  Version:          1.2.5
  GitCommit:        v1.2.5-0-g59923ef
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker pull nginx
docker run -d -p 8080:80 nginx
Using default tag: latest
latest: Pulling from library/nginx
700146c8ad64: Pull complete
10b68cfefee1: Pull complete
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
57f0dd1befe2: Pull complete
```

Observation:

Docker successfully downloaded the nginx image layers from Docker Hub and stored them locally.

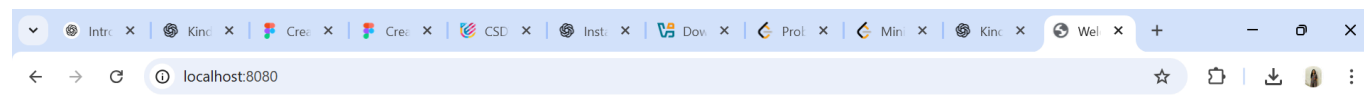
Step 2: Run Container with Port Mapping

A container was created and run in detached mode with port mapping.

```
docker run -d -p 8080:80 nginx
```

```
Select harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab
containerd:
  Version:      1.7.27
  GitCommit:    05044ec0a9a75232cad458027ca83437aae3f4da
runc:
  Version:      1.2.5
  GitCommit:    v1.2.5-0-g59923ef
docker-init:
  Version:      0.19.0
  GitCommit:    de40ad0
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ docker pull nginx
docker run -d -p 8080:80 nginx
Using default tag: latest
latest: Pulling from library/nginx
700146c8ad64: Pull complete
10b68cfeae1: Pull complete
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
57f0dd1befe2: Pull complete
119d43eac815: Pull complete
500799c30424: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
f76031f996a9ed147038793916dcb165cd26c6123d83401256617d299b8e007c
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ docker run -d -p 8080:80 nginx
5ad3f18b1bc470c2b92a0338a634c30bef7cfc85d6125225bdd42f2ad3274534
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint gallant_mcnulty (0d1724c706b90bd2e446
92ad403f82b56514c699e08e23e526d407224aec37d): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f76031f996a9   nginx    "/docker-entrypoint..." 4 minutes ago  Up 4 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  bold_joliot
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ ^[[200-docker stop f76031f996a9
docker rm f76031f996a9
^[[201-docker: command not found
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ docker rm f76031f996a9
Error response from daemon: cannot remove container "f76031f996a9": container is running: stop the container before removing or force remove
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ ~
-bash: /home/harshita24: Is a directory
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ docker rmi nginx
Error response from daemon: conflict: unable to delete nginx:latest (must be forced) - container f76031f996a9 is using its referenced image c881927c4077
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$
```



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org. Commercial support is available at nginx.com.

Thank you for using nginx.

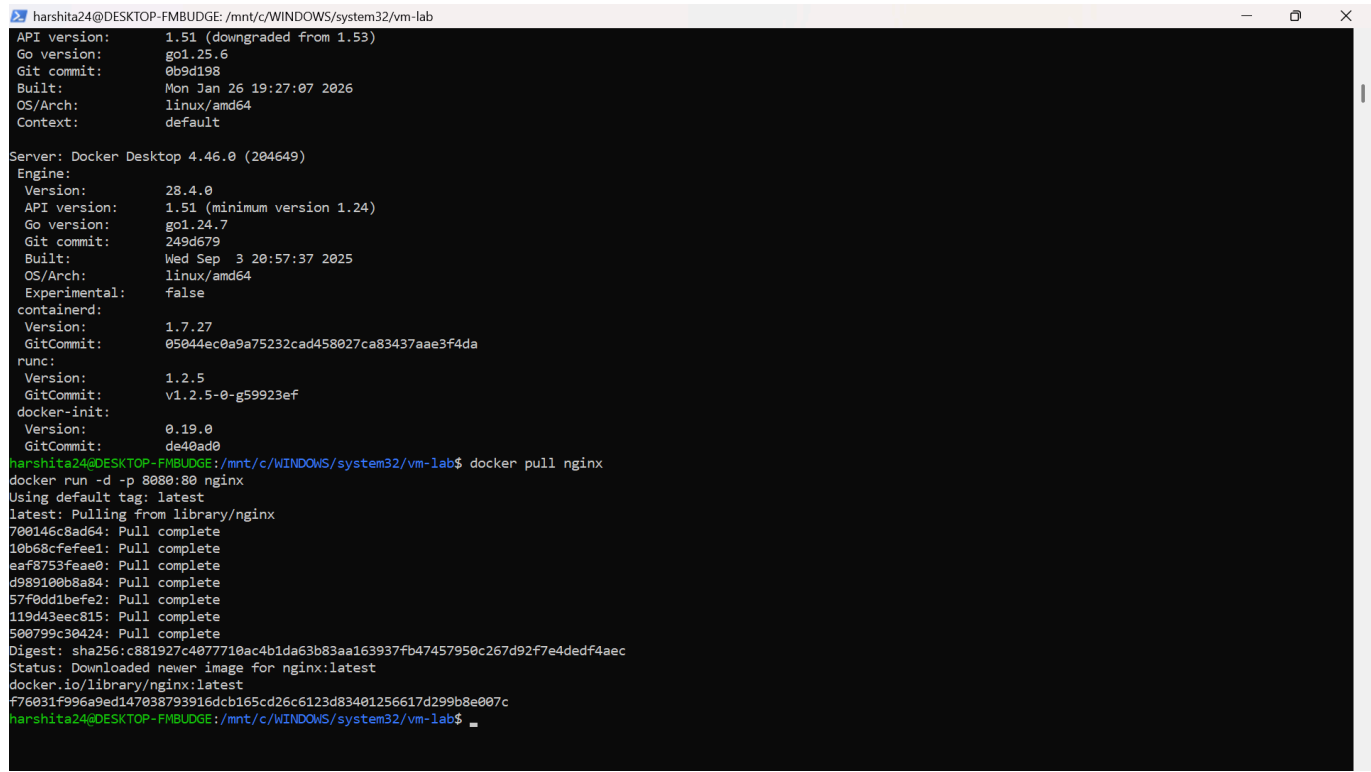
Observation:

The container started successfully in detached mode. Port 8080 on the host machine was mapped to port 80 inside the container.

Step 3: Verify Running Containers

To verify the running containers, the following command was used:

```
docker ps
```



```
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab
API version: 1.51 (downgraded from 1.53)
Go version: go1.25.6
Git commit: 0b9d198
Built: Mon Jan 26 19:27:07 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Desktop 4.46.0 (204649)
Engine:
  Version: 28.4.0
  API version: 1.51 (minimum version 1.24)
  Go version: go1.24.7
  Git commit: 249d679
  Built: Wed Sep 3 20:57:37 2025
  OS/Arch: linux/amd64
  Experimental: false
containerd:
  Version: 1.7.27
  GitCommit: 05044ec0a9a75232cad458027ca83437aae3f4da
runc:
  Version: 1.2.5
  GitCommit: v1.2.5-0-g59923ef
docker-init:
  Version: 0.19.0
  GitCommit: de40ad0

harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$ docker pull nginx
docker run -d -p 8080:80 nginx
Using default tag: latest
latest: Pulling from library/nginx
700146c8ad64: Pull complete
10b68cfefae1: Pull complete
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
57f0dd1befe2: Pull complete
119d43eec815: Pull complete
500799c30424: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
f76031f996a9ed147038793916dcb165cd26c6123d83401256617d299b8e007c
harshita24@DESKTOP-FMBUDGE:/mnt/c/WINDOWS/system32/vm-lab$
```

Observation:

The nginx container was listed as running with its container ID, image name, and mapped ports.

Step 4: Stop and Remove Container

To stop the running container:

```
docker stop <container_id>
```

To remove the container:

```
docker rm <container_id>
```

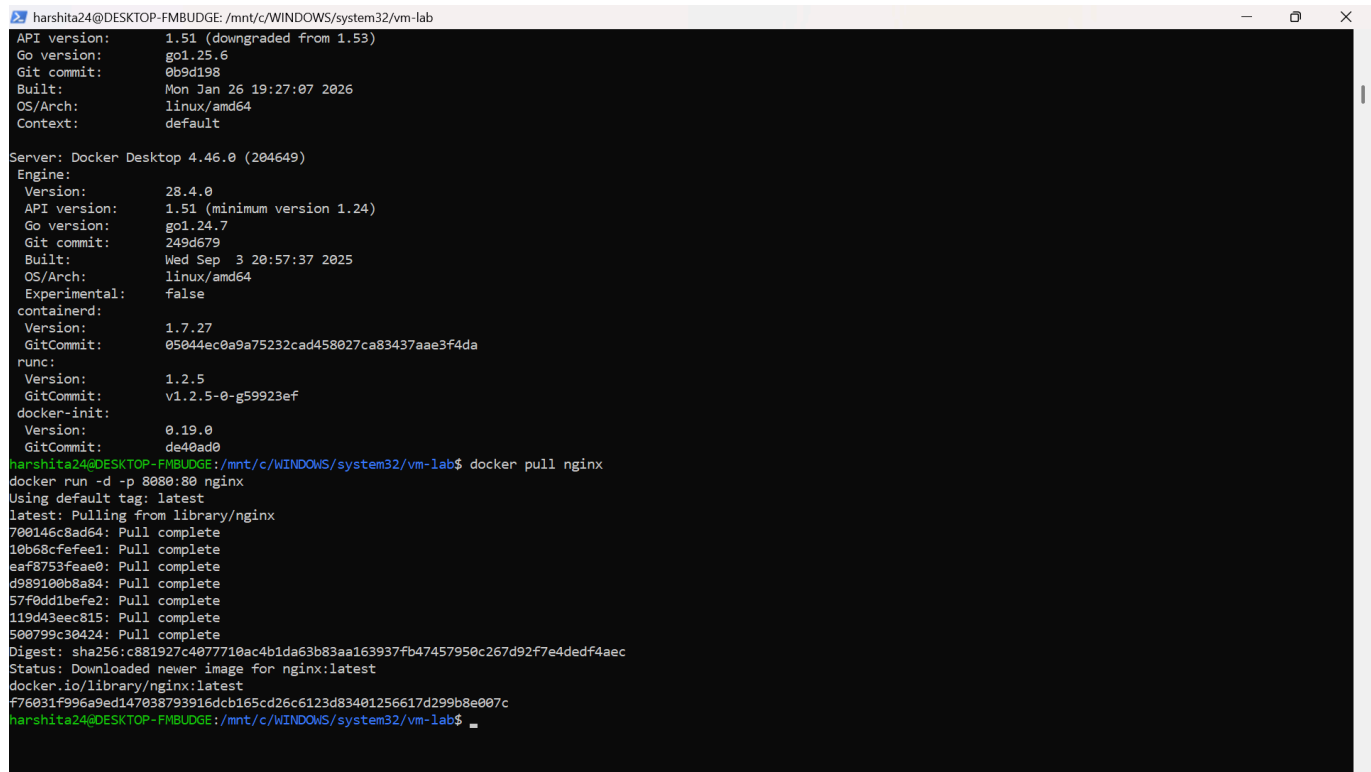
Observation:

The container was successfully stopped and removed from the system.

Step 5: Remove Docker Image

To remove the nginx image:

```
docker rmi nginx
```



```
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab
API version:      1.51 (downgraded from 1.53)
Go version:       go1.25.6
Git commit:       0b9d198
Built:           Mon Jan 26 19:27:07 2026
OS/Arch:         linux/amd64
Context:         default

Server: Docker Desktop 4.46.0 (204649)
Engine:
  Version:        28.4.0
  API version:    1.51 (minimum version 1.24)
  Go version:     go1.24.7
  Git commit:     249d679
  Built:         Wed Sep 3 20:57:37 2025
  OS/Arch:       linux/amd64
  Experimental:   false
containerd:
  Version:        1.7.27
  GitCommit:      05044ec0a9a75232cad458027ca83437aae3f4da
runc:
  Version:        1.2.5
  GitCommit:      v1.2.5-0-g59923ef
docker-init:
  Version:        0.19.0
  GitCommit:      de40ad0
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$ docker pull nginx
docker run -d -p 8080:80 nginx
Using default tag: latest
latest: Pulling from library/nginx
700146c8ad64: Pull complete
10b68cfefee1: Pull complete
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
57f0dd1befe2: Pull complete
119d43eec815: Pull complete
500799c30424: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
f76031f996a9ed14708793916dcb165cd26c6123d83401256617d299b8e007c
harshita24@DESKTOP-FMBUDGE: /mnt/c/WINDOWS/system32/vm-lab$
```

Observation:

The nginx image was removed from the local Docker image repository.

3. Result

Docker images were successfully pulled from Docker Hub.

Containers were created, executed, verified, stopped, and removed.

The container lifecycle management commands were performed successfully.

Lab 3: Deploying NGINX Using Different Base Images and Comparing Image Layers

Lab Objectives

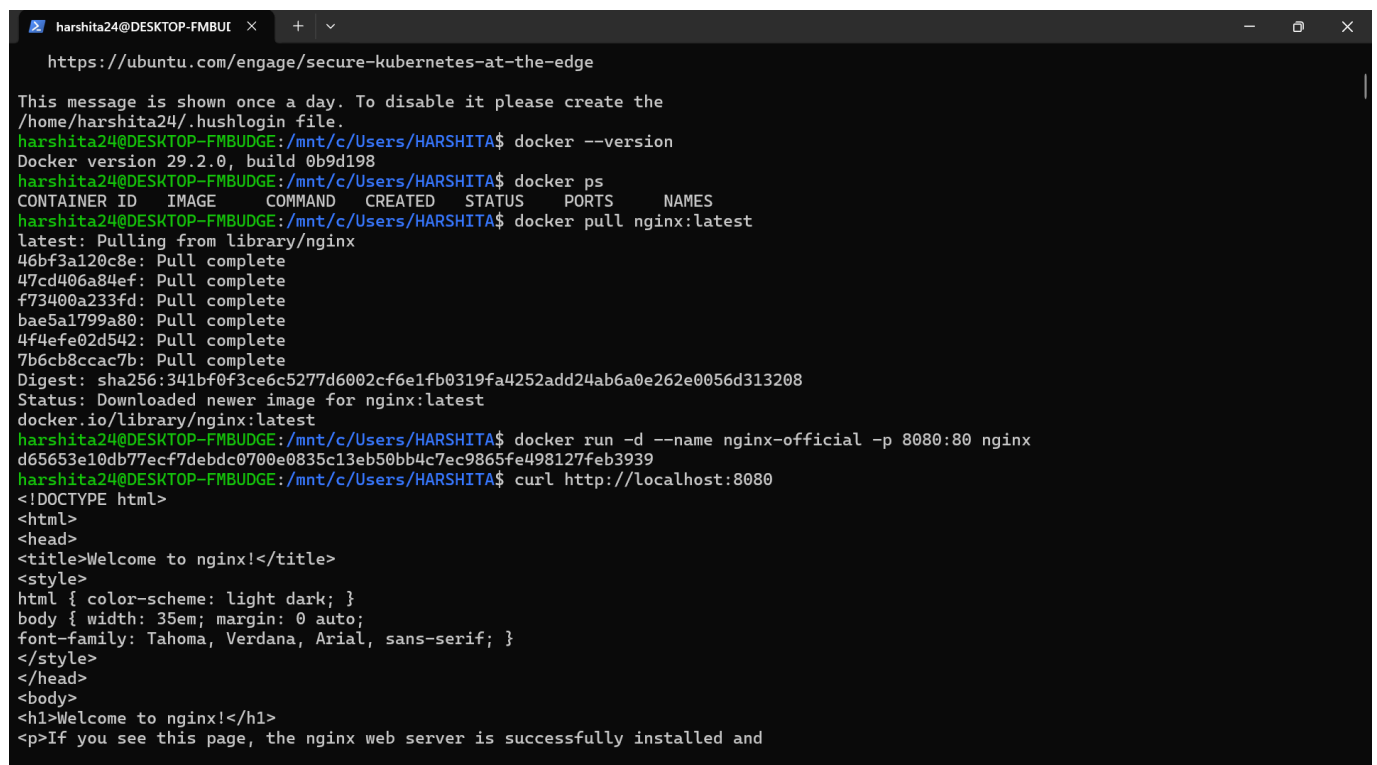
After completing this lab, students will be able to:

- Deploy NGINX using:
 - Official NGINX image
 - Ubuntu-based image
 - Alpine-based image
- Understand Docker image layers and size differences
- Compare performance, security, and use-cases
- Explain real-world use of NGINX in containerized systems

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

Step 1: Pull the Image

```
docker pull nginx:latest
```



```
harshita24@DESKTOP-FMBUI: ~$ docker --version
Docker version 29.2.0, build 0b9d198
harshita24@DESKTOP-FMBUI: ~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
harshita24@DESKTOP-FMBUI: ~$ docker pull nginx:latest
latest: Pulling from library/nginx
46bf3a120c8e: Pull complete
47cd406a84ef: Pull complete
f73400a233fd: Pull complete
bae5a1799a80: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
harshita24@DESKTOP-FMBUI: ~$ docker run -d --name nginx-official -p 8080:80 nginx
d65653e10db77ecf7debd0700e0835c13eb50bb4c7ec9865fe498127feb3939
harshita24@DESKTOP-FMBUI: ~$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
```

Step 2: Run the Container

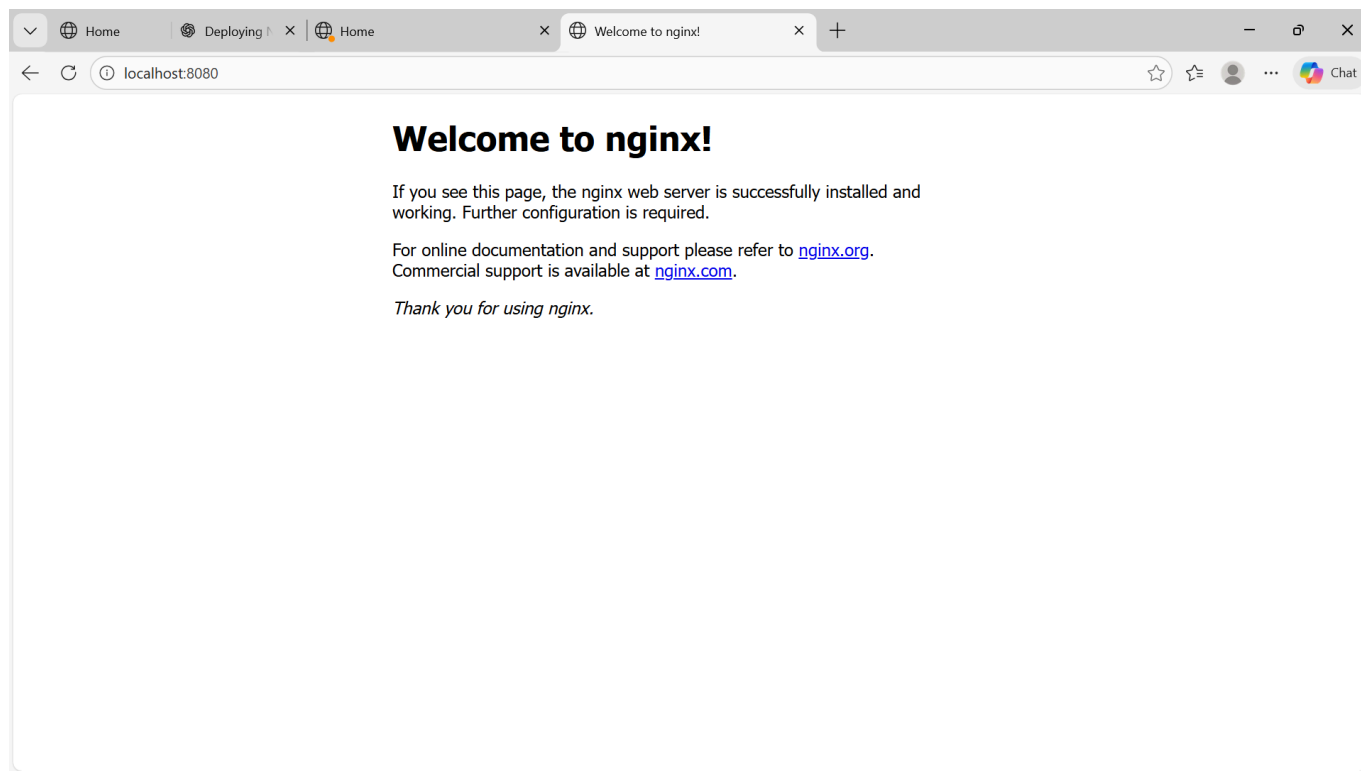
```
docker run -d --name nginx-official -p 8080:80 nginx
```

```
harshita24@DESKTOP-FMBUI x + v
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker images nginx
INFO → U In Use
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
nginx:latest    341bf0f3ce6c      237MB       62.9MB        U
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ mkdir nginx-ubuntu
cd nginx-ubuntu
nano Dockerfile
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-ubuntu$ docker build -t nginx-ubuntu .
[+] Building 67.5s (6/6) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 218B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04  4.7s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706 13.3s
=> => resolve docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706c 0.0s
=> => sha256:6f4ebca3e823b18dac366f72e537b1772bc3522a5c7ae299d6491fb17378410e 29.54MB / 29.54MB 11.9s
=> => extracting sha256:6f4ebca3e823b18dac366f72e537b1772bc3522a5c7ae299d6491fb17378410e 1.2s
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lis 45.2s
=> exporting to image                                           3.7s
=> => exporting layers                                           2.8s
=> => exporting manifest sha256:047c229a0f0e8429351df2ae6113d79ea8fc7d2d57a7c058fd974af3bc704999 0.0s
=> => exporting config sha256:3ffa609950c72b48d328b5a083304e0f5455dc1d91531a2deb4665280b92ad9b 0.0s
=> => exporting attestation manifest sha256:722db8bf3725833e590a80985de3510dff4b5e83ff9e9ae11e4ccf633c8a899c 0.0s
=> => exporting manifest list sha256:bd1b6f11325d6b168772c3e0c45980f6cc40e27b7a5642c46d71e19c1a1d6e1b 0.0s
=> => naming to docker.io/library/nginx-ubuntu:latest          0.0s
=> => unpacking to docker.io/library/nginx-ubuntu:latest        0.8s
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-ubuntu$ docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
05e294a582c5bf8d9660913ee88441c9dd1922551835f9e8463d1cc2187beb97
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-ubuntu$ curl http://localhost:8081
<!DOCTYPE html>
<html>
```

Step 3: Verify

```
curl http://localhost:8080
```

You should see the NGINX welcome page.



Key Observations

docker images nginx

```
harshita24@DESKTOP-FMBUI x + v
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-ubuntu$ docker images nginx-ubuntu
INFO → U In Use
IMAGE          ID          DISK USAGE  CONTENT SIZE  EXTRA
nginx-ubuntu:latest bd1b6f11325d 199MB       52.7MB        U
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-ubuntu$ exit
logout
PS C:\Users\HARSHITA> wsl
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ mkdir ../nginx-alpine
cd ../nginx-alpine
nano Dockerfile
mkdir: cannot create directory '../nginx-alpine': Permission denied
-bash: cd: ../nginx-alpine: No such file or directory
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker build -t nginx-alpine .
[+] Building 18.4s (6/6) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 133B                               0.1s
=> [internal] load metadata for docker.io/library/alpine:latest   7.3s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                       0.1s
=> [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c 5.2s
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c 0.0s
=> => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 3.86MB / 3.86MB 4.9s
=> => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153 0.2s
=> [2/2] RUN apk add --no-cache nginx                             4.7s
=> => exporting to image                                           0.6s
=> => exporting layers                                             0.3s
=> => exporting manifest sha256:7a98bf24a58fbef28874e6dd5a9091a6529e4aa3ef0e761d75df3baa7a472954 0.0s
=> => exporting config sha256:0aa75b70f081aced8c8b292b0605a16ad9c0138d5da4e5276165aa9557b6a2c6 0.0s
=> => exporting attestation manifest sha256:0ab7f96397b7a62e5fba940d948824a61d1dbab227d21011a0ae8e5b7f42ef19 0.1s
=> => exporting manifest list sha256:ab875f1be07134542466762e14618b5ec5e10da10ff8f4fe1d03ae5031011d14 0.0s
=> => naming to docker.io/library/nginx-alpine:latest            0.0s
=> => unpacking to docker.io/library/nginx-alpine:latest          0.1s
```

- Image is pre-optimized
- Minimal configuration required
- Uses Debian-based OS internally

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

Create a file named `Dockerfile` and add:

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

```
harshita24@DESKTOP-FMBUI x + v
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-alpine$ docker images | grep nginx
docker ps
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx-alpine:latest      c14e2f8b15cc      16MB      4.74MB      U
nginx-ubuntu:latest      bd1b6f11325d      199MB     52.7MB      U
nginx:latest             341bf0f3ce6c      237MB     62.9MB      U
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4fcfe12c6d21   nginx-alpine   "nginx -g 'daemon of..." 48 seconds ago Up 48 seconds 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  ngi
nx-alpine
05e294a582c5   nginx-ubuntu   "nginx -g 'daemon of..." 15 minutes ago Up 15 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  ngi
nx-ubuntu
d65653e10db7   nginx          "/docker-entrypoint..." 18 minutes ago Up 18 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  ngi
nx-official
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-alpine$ docker rm -f nginx-alpine
nginx-alpine
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/nginx-alpine$ exit
logout
PS C:\Users\HARSHITA> wsl
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ mkdir alpine-html
echo "<h1>Hello from Alpine NGINX</h1>" > alpine-html/index.html
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
--name nginx-alpine \
-p 8082:80 \
-v $(pwd)/alpine-html:/usr/share/nginx/html \
nginx-alpine
51e5011e2804022531f0f8bfd70dd61713097f6b3823e365220a851b3a03c46e
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ curl http://localhost:8082
```

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

Observations

```
docker images nginx-ubuntu
```

- Much larger image size
 - More layers
 - Full OS utilities available
-

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

Observations


```
docker images nginx-alpine
```

```
harshita24@DESKTOP-FMBUI x + v
f8bfd70dd61713097f6b3823e365220a851b3a03c46e". You have to remove (or rename) that container to be able to reuse that name.
Run 'docker run --help' for more information
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker images nginx-alpine
INFO In Use
IMAGE          ID          DISK USAGE  CONTENT SIZE  EXTRA
nginx-alpine:latest  9a0f530e4bc8  16MB        4.74MB
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker images | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx-alpine:latest  9a0f530e4bc8  16MB        4.74MB
nginx-ubuntu:latest  e448bb3243e8  16MB        4.74MB
nginx:latest        341bf0f3ce6c  237MB       62.9MB       U
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE          CREATED      CREATED BY          SIZE      COMMENT
341bf0f3ce6c   2 days ago  CMD ["nginx" "-g" "daemon off;"]  0B        buildkit.dockerfile.v0
<missing>     2 days ago  STOPSIGNAL SIGQUIT              0B        buildkit.dockerfile.v0
<missing>     2 days ago  EXPOSE map[80/tcp:{}]           0B        buildkit.dockerfile.v0
<missing>     2 days ago  ENTRYPOINT ["/docker-entrypoint.sh"] 0B        buildkit.dockerfile.v0
<missing>     2 days ago  COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB    buildkit.dockerfile.v0
<missing>     2 days ago  COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB    buildkit.dockerfile.v0
<missing>     2 days ago  COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB    buildkit.dockerfile.v0
<missing>     2 days ago  COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB    buildkit.dockerfile.v0
<missing>     2 days ago  COPY docker-entrypoint.sh / # buildkit 8.19kB    buildkit.dockerfile.v0
<missing>     2 days ago  RUN /bin/sh -c set -x && groupadd --syst... 86.6MB    buildkit.dockerfile.v0
<missing>     2 days ago  ENV DYNPKG_RELEASE=1~trixie      0B        buildkit.dockerfile.v0
<missing>     2 days ago  ENV PKG_RELEASE=1~trixie        0B        buildkit.dockerfile.v0
<missing>     2 days ago  ENV ACME_VERSION=0.3.1          0B        buildkit.dockerfile.v0
<missing>     2 days ago  ENV NJS_RELEASE=1~trixie        0B        buildkit.dockerfile.v0
<missing>     2 days ago  ENV NJS_VERSION=0.9.5           0B        buildkit.dockerfile.v0
<missing>     2 days ago  ENV NGINX_VERSION=1.29.5        0B        buildkit.dockerfile.v0
<missing>     2 days ago  LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>     5 days ago  # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB    debuerreotype 0.17
IMAGE          CREATED      CREATED BY          SIZE      COMMENT
```

- Extremely small image
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

Typical Size Comparison

Image Type	Approx Size
nginx:latest	~140 MB
nginx-ubuntu	~220+ MB
nginx-alpine	~25-30 MB

Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

```
harshita24@DESKTOP-FMBUI x + v
f8bfd70dd61713097f6b3823e365220a851b3a03c46e". You have to remove (or rename) that container to be able to reuse that name.
Run 'docker run --help' for more information
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker images nginx-alpine
Info → U In Use
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
nginx-alpine:latest 9a0f530e4bc8      16MB        4.74MB
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker images | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx-alpine:latest 9a0f530e4bc8      16MB        4.74MB
nginx-ubuntu:latest e448bb3243e8      16MB        4.74MB
nginx:latest       341bf0f3ce6c      237MB       62.9MB       U
harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
341bf0f3ce6c    2 days ago      CMD ["nginx" "-g" "daemon off;"]              0B         buildkit.dockerfile.v0
<missing>       2 days ago      STOPSIGNAL SIGQUIT                             0B         buildkit.dockerfile.v0
<missing>       2 days ago      EXPOSE map[80/tcp:{}]                          0B         buildkit.dockerfile.v0
<missing>       2 days ago      ENTRYPOINT ["/docker-entrypoint.sh"]           0B         buildkit.dockerfile.v0
<missing>       2 days ago      COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB     buildkit.dockerfile.v0
<missing>       2 days ago      COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB     buildkit.dockerfile.v0
<missing>       2 days ago      COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB     buildkit.dockerfile.v0
<missing>       2 days ago      COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB     buildkit.dockerfile.v0
<missing>       2 days ago      COPY docker-entrypoint.sh / # buildkit         8.19kB     buildkit.dockerfile.v0
<missing>       2 days ago      RUN /bin/sh -c set -x && groupadd --syst...     86.6MB     buildkit.dockerfile.v0
<missing>       2 days ago      ENV DYNPKG_RELEASE=1~trixie                     0B         buildkit.dockerfile.v0
<missing>       2 days ago      ENV PKG_RELEASE=1~trixie                         0B         buildkit.dockerfile.v0
<missing>       2 days ago      ENV ACME_VERSION=0.3.1                          0B         buildkit.dockerfile.v0
<missing>       2 days ago      ENV NJS_RELEASE=1~trixie                         0B         buildkit.dockerfile.v0
<missing>       2 days ago      ENV NJS_VERSION=0.9.5                           0B         buildkit.dockerfile.v0
<missing>       2 days ago      ENV NGINX_VERSION=1.29.5                        0B         buildkit.dockerfile.v0
<missing>       2 days ago      LABEL maintainer=NGINX Docker Maintainers <d... 0B         buildkit.dockerfile.v0
<missing>       5 days ago      # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB     debuerreotype 0.17
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
```

Observations

- Ubuntu image has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Task Using NGINX

Task 1: Serve Custom HTML Page

```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

Run container with volume mounting:

```
docker run -d \
-p 8083:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx
```

Result

NGINX was successfully deployed using three different base images. Image sizes, layers, and performance characteristics were compared. Custom HTML content was served using volume mounting.

Experiment 3: Deploying Web Applications with Docker

In this experiment, we deploy a sample web application using Docker. This includes writing a custom Dockerfile, building a Docker image, and running the application inside a containerized environment.

This experiment demonstrates the complete lifecycle of containerizing and deploying a web application.

Objectives

By completing this experiment, we will:

1. Prepare a simple web application for containerization.
 2. Write a Dockerfile defining the application environment.
 3. Build a custom Docker image.
 4. Run the web app inside a Docker container.
 5. Verify that the application works correctly in the container.
-

Step 1: Choose a Sample Application

We selected a lightweight Python Flask web application.

Sample Flask Application (app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, Docker!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

```

harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA$ mkdir flask-docker-app
cd flask-docker-app
harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ nano app.py
harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ pip3 install flask
Command 'pip3' not found, but can be installed with:
sudo apt install python3-pip
harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ sudo apt install python3-pip
[sudo] password for harshita24:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg dpkg-dev fakeroot g++ g++-13
  g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu
  javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libaom3 libasan8 libatomic1 libc-bin
  libc-dev-bin libc-devtools libc6 libc6-dev libc6-i386 libcrypt-dev libdeb265-0 libdpkg-perl libexpat1-dev libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgd3 libgomp1 libheif-plugin-aomdec libheif-plugin-aomenc libheif-plugin-libde265 libheif1
  libhwasan0 libisl23 libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore liblsan0 libmpc3 libpython3-dev libpython3-stdlib
  libpython3.12-dev libpython3.12-minimal libpython3.12-stdlib libpython3.12t64 libquadmath0 libstdc++-13-dev libtsan2 libubsan1
  libxpm4 linux-libc-dev locales lto-disabled-list make manpages-dev python3 python3-dev python3-minimal python3-wheel python3.12
  python3.12-dev python3.12-minimal rpcsvc-proto zlib1g-dev
Suggested packages:
  bzip2-doc cpp-doc gcc-13-locales cpp-13-doc debsig-verify debian-keyring g++-multilib g++-13-multilib gcc-13-doc gcc-multilib
  autoconf automake libtool flex bison gdb gcc-doc gcc-13-multilib gdb-x86-64-linux-gnu glibc-doc libnss-nis libnss-nisplus bsr
  libgd-tools libheif-plugin-x265 libheif-plugin-ffmpegdec libheif-plugin-jpegdec libheif-plugin-jpegenc libheif-plugin-j2kdec
  libheif-plugin-j2kenc libheif-plugin-rav1e libheif-plugin-svtenc libstdc++-13-doc make-doc python3-doc python3-tk python3-venv
The following NEW packages will be installed:
  build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot g++ g++-13
  g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu
  javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libaom3 libasan8 libatomic1
  libc-dev-bin libc-devtools libc6-dev libc6-i386 libcrypt-dev libdeb265-0 libdpkg-perl libexpat1-dev libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgd3 libgomp1 libheif-plugin-aomdec libheif-plugin-aomenc libheif-plugin-libde265 libheif1
  libhwasan0 libisl23 libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore liblsan0 libmpc3 libpython3-dev libpython3.12-dev
  libquadmath0 libstdc++-13-dev libtsan2 libubsan1 libxpm4 linux-libc-dev lto-disabled-list make manpages-dev python3-dev

```

```

Processing triggers for fontconfig (2.15.0-1.lubuntu2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ /mnt/c/Users/HARSHITA/flask-docker-app
-bash: /mnt/c/Users/HARSHITA/flask-docker-app: Is a directory
harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ python3 -m venv venv

source venv/bin/activate

harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$
harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ source venv/bin/activate
(venv) harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$
(venv) harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$
(venv) harshita24@DESKTOP-FMBUI: /mnt/c/Users/HARSHITA/flask-docker-app$ pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.2-py3-none-any.whl (103 kB)
103.3/103.3 kB 184.0 kB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)

```

Step 2: Create Application Files

Inside the project directory, create:

- `app.py`
- `requirements.txt`

`requirements.txt`

flask

```

harshita24@DESKTOP-FMBUI x + v
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.2-py3-none-any.whl (103 kB)
103.3/103.3 kB 184.0 kB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
108.3/108.3 kB 105.3 kB/s eta 0:00:00
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
134.9/134.9 kB 90.9 kB/s eta 0:00:00
Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
225.0/225.0 kB 74.8 kB/s eta 0:00:00
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.1 flask-3.1.2 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.5
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.22.223.109:5000

```

```

harshita24@DESKTOP-FMBUI x + v
* Running on http://172.22.223.109:5000
Press CTRL+C to quit
127.0.0.1 - - [07/Feb/2026 09:46:40] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Feb/2026 09:46:40] "GET /favicon.ico HTTP/1.1" 404 -
^C(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ nano app.py
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ nano requirements.txt
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ FROM python:3.9-slim
FROM: command not found
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ nano Dockerfile
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ ls
Dockerfile app.py requirements.txt nano
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker build -t flask-web-app .
[+] Building 27.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 204B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> => sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 251B / 251B
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 1.4s
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 0.2s
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 0.8s
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 0.0s
=> [internal] load build context
=> => transferring context: 258B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> exporting to image
docker:default
0.1s
0.0s
6.6s
0.1s
0.0s
9.6s
0.1s
4.4s
1.9s
1.8s
6.8s
1.4s
0.2s
0.8s
0.0s
0.1s
0.0s
0.3s
0.1s
8.6s
0.1s
1.5s

```

Step 3: Test the Application Locally

Before containerizing, test locally:

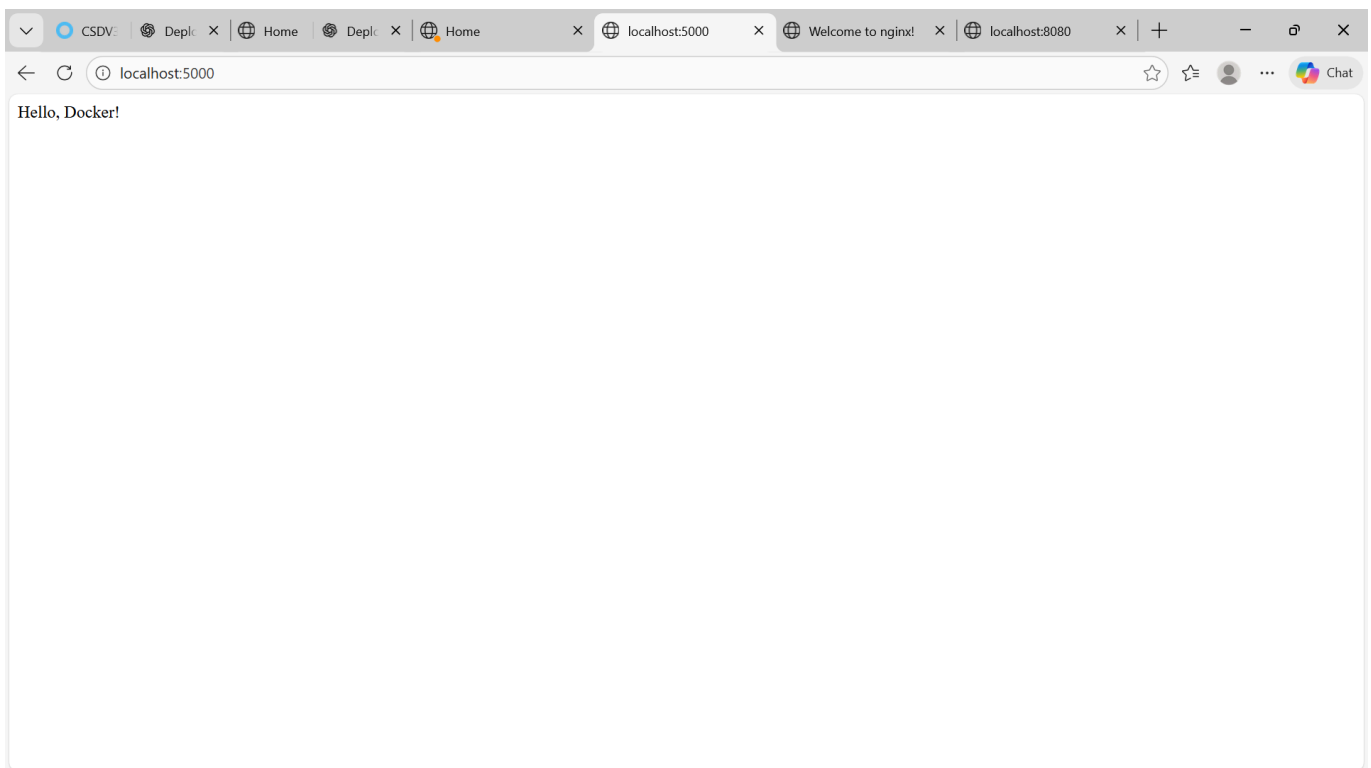
```
python app.py
```

Visit:

```
http://localhost:8080
```

Expected Output:

```
Hello, Docker!
```



Fix any errors before proceeding.

Step 4: Writing the Dockerfile

Create a file named **Dockerfile**.

Dockerfile Content

```
# Base Image
FROM python:3.9-slim

# Set working directory
```

```
WORKDIR /app

# Copy application files
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose port
EXPOSE 8080

# Start application
CMD ["python", "app.py"]
```



```

harshita24@DESKTOP-FMBUI x + v
=> => naming to docker.io/library/flask-web-app:latest 0.0s
=> => unpacking to docker.io/library/flask-web-app:latest 0.4s
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 5000:5000 --name flask-container flask-web-app
78cladb5e747f389924c4d3658f23bc15537521814f88fe1784089b2d6d77f94
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
78cladb5e747f flask-web-app   "python app.py"         15 seconds ago Up 13 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
533ba8ec18ce   nginx         "/docker-entrypoint..." 27 minutes ago Up 27 minutes 0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
51e5011e2804   c14e2f8b15cc  "nginx -g 'daemon of..." 32 minutes ago Up 32 minutes 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
05e294a582c5   bd1b6f11325d  "nginx -g 'daemon of..." 50 minutes ago Up 50 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
d65653e10db7   nginx         "/docker-entrypoint..." 53 minutes ago Up 53 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker logs myflaskcontainer
Error response from daemon: No such container: myflaskcontainer
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker build -t myflaskapp .
[+] Building 1.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 204B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b 0.1s
=> [internal] load build context 0.1s
=> => transferring context: 63B 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt . 0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py . 0.0s

```

```

harshita24@DESKTOP-FMBUI x + v
Error response from daemon: No such container: myflaskcontainer
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker build -t myflaskapp .
[+] Building 1.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 204B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b 0.1s
=> [internal] load build context 0.1s
=> => transferring context: 63B 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt . 0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py . 0.0s
=> exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:abf138dea980a819024e2fcl055a8d23e9e2ee277ee2dbc11e99a347f0fbb54 0.0s
=> => exporting config sha256:44273e0e307f9903f1b19cfac60ae06078fcb72f4c0d0491bb6e2a5f28b9cb3 0.0s
=> => exporting attestation manifest sha256:1296806f98510ec03d90dcfda9382650b27aa42af2864ec9733631b386f88d19 0.1s
=> => exporting manifest list sha256:0ecdea91dd5075f98b6e1e46000ef450e5ea0c774acfd6c5b6add88bf4a4997 0.0s
=> => naming to docker.io/library/myflaskapp:latest 0.0s
=> => unpacking to docker.io/library/myflaskapp:latest 0.0s
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 5000:5000 --name myflaskcontainer myflaskapp
fb2b87137f848e2a484e330f6fc3184b1f0191c7e2e987f8787e01663c191ca8
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint myflaskcontainer (b08bb90f1455b8de1d3be7b292b5f63e42ec1743a26155a834234cdcd1f75d5): Bind for 0.0.0.0:5000 failed: port is already allocated

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
docker: Error response from daemon: Conflict. The container name "/myflaskcontainer" is already in use by container "fb2b87137f848e2a

```

Explanation of Dockerfile Sections

1. Base Image

FROM python:3.9-slim

This provides a lightweight Python runtime environment.

2. Copying Files


```
COPY . /app
```

Copies all application files into the container.

3. Installing Dependencies

```
RUN pip install --no-cache-dir -r requirements.txt
```

Installs Flask and other dependencies.

4. Entrypoint / Command

```
CMD ["python", "app.py"]
```

Defines the default command to run the application.

Step 5: Build the Docker Image

Run:

```
docker build -t flask-docker-app .
```

```

harshita24@DESKTOP-FMBUI x + v
Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
docker: Error response from daemon: Conflict. The container name "/myflaskcontainer" is already in use by container "fb2b87137f848e2a484e330f6fc3184b1f0191c7e2e987f8787e01663c191ca8". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f myflaskcontainer
myflaskcontainer
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
f505e4930ae292cdcbab374b3b30f7c1a8f5a0dae9ad1355b4a911b333b0f947
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint myflaskcontainer (b3f6ef68b4bf44c3d8cb4f16cb78c3d04cba587f76f98c81eaea98d6cbd9368f): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ (venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
f505e4930ae292cdcbab374b3b30f7c1a8f5a0dae9ad1355b4a911b333b0f947
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint myflaskcontainer (b3f6ef68b4bf44c3d8cb4f16cb78c3d04cba587f76f98c81eaea98d6cbd9368f): Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$
-bash: syntax error near unexpected token `harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$'

f505e4930ae292cdcbab374b3b30f7c1a8f5a0dae9ad1355b4a911b333b0f947: command not found
-bash: syntax error near unexpected token `('
Command 'Run' not found, did you mean:
  command 'bun' from snap bun-js (1.3.8)
  command 'zun' from deb python3-zunclient (4.7.0-0ubuntu1)
See 'snap info <snapname>' for additional versions.

```

```

harshita24@DESKTOP-FMBUI x + v
See 'snap info <snapname>' for additional versions.
-bash: syntax error near unexpected token `harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$'
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
78cladb5e747   flask-web-app  "python app.py"         4 minutes ago Up 4 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
533ba8ec18ce   nginx         "/docker-entrypoint..." 32 minutes ago Up 32 minutes  0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
sad_blackburn  sad_blackburn "nginx -g 'daemon of..." 37 minutes ago Up 37 minutes  0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
51e5011e2804   nginx-alpine  "nginx -g 'daemon of..." 54 minutes ago Up 54 minutes  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
05e294a582c5   bd1b6f11325d  "nginx -g 'daemon of..." 57 minutes ago Up 57 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
d65653e10db7   nginx-official
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f nginx-official
nginx-official
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
pp
docker: Error response from daemon: Conflict. The container name "/myflaskcontainer" is already in use by container "f505e4930ae292cdcbab374b3b30f7c1a8f5a0dae9ad1355b4a911b333b0f947". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f flask-container
flask-container
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
pp
docker: Error response from daemon: Conflict. The container name "/myflaskcontainer" is already in use by container "f505e4930ae292cdcbab374b3b30f7c1a8f5a0dae9ad1355b4a911b333b0f947". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f myflaskcontainer
myflaskcontainer
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f nginx-official

```

Step 6: Run the Docker Container

```
docker run -d -p 8080:8080 --name flask-container flask-docker-app
```

```
harshita24@DESKTOP-FMBUI x + v
flask-container
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
docker: Error response from daemon: Conflict. The container name "/myflaskcontainer" is already in use by container "f505e4930ae292cdcbab374b3b30f7c1a8f5a0dae9ad1355b4a911b333b0f947". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f myflaskcontainer
myflaskcontainer
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker rm -f nginx-official
Error response from daemon: No such container: nginx-official
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker run -d -p 8080:5000 --name myflaskcontainer myflaskapp
8780c4b5efa0336217cb1381df59925f0e7c515dee39781a62ecc92a6d409d0c
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$ docker ps
docker logs myflaskcontainer
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
8780c4b5efa0   myflaskapp     "python app.py"         32 seconds ago Up 31 seconds 0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp
myflaskcontainer
533ba8ec18ce   nginx          "/docker-entrypoint..." 34 minutes ago Up 34 minutes 0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
sad_blackburn
51e5011e2804   c14e2f8b15cc   "nginx -g 'daemon of..." 39 minutes ago Up 39 minutes 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
nginx-alpine
05e294a582c5   bd1b6f11325d   "nginx -g 'daemon of..." 56 minutes ago Up 56 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
nginx-ubuntu
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [07/Feb/2026 10:00:11] "GET / HTTP/1.1" 200 -
(venv) harshita24@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-docker-app$
```

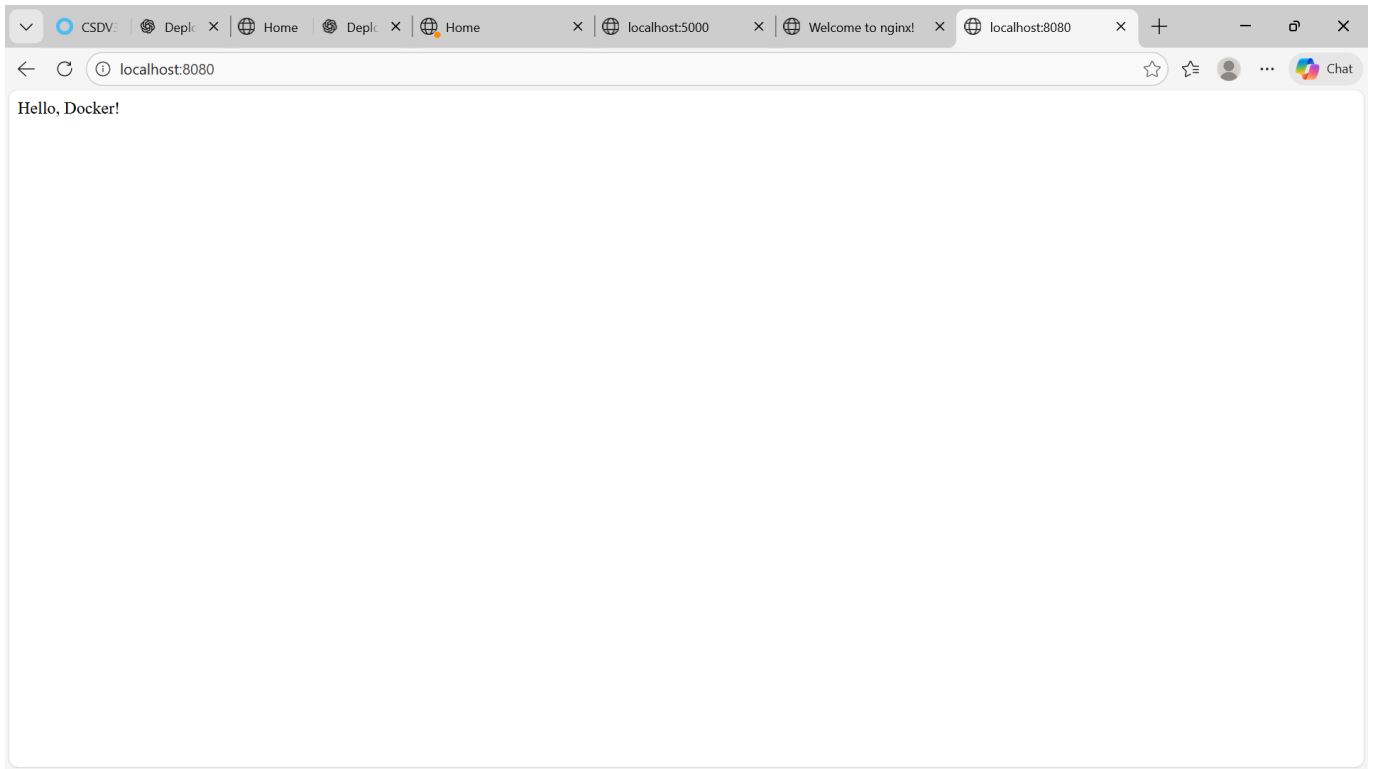
Step 7: Verify the Application

Open in browser:

```
http://localhost:8080
```

Expected Output:

```
Hello, Docker!
```



Troubleshooting (If Issues Occur)

If nothing loads:

- Check container status:

```
docker ps
```

- Check container logs:

```
docker logs flask-container
```

Result

The web application was successfully containerized using Docker.

A custom image was built, the container was launched, and the application was accessed via a web browser.

This experiment demonstrates the complete workflow of deploying a web application in a containerized environment.

. dockerignore

Experiment 4: Docker Essentials

Dockerfile

Part 1: Containerizing Applications with Dockerfile

Step 1: Create a Simple Application

Python Flask App:

tagging

publishing

```
mkdir my-flask-app cd my-flask-app
```

```
app.py from flask import Flask app = Flask(__name__
```

```
@app.route('/') def hello(): return "Hello from Docker!"
```

```
@app.route('/health') def health(): return "OK"
```

```
name app.run(host='0.0.0.0', port=5000)
```

if

main

requirements.txt

```
Flask == 2.3.3
```

Step 2: Create Dockerfile Dockerfile

Use Python base image

```
FROM python: 3.9-slim
```

Set working directory

```
WORKDIR /app
```

Copy requirements file

```
COPY requirements.txt .
```

Install dependencies

RUN pip install -- no-cache-dir -r requirements.txt

Copy application code

COPY app.py .

Expose port

EXPOSE 5000

Run the application

CMD ["python", "app.py"] Part 2: Using .dockerignore

Step 1: Create .dockerignore File

. dockerignore

.

Python files

_pycache

- . pyc
- . pyo .pyd

Environment files

. env . venv env/ venv/

IDE files

. vscode/ .idea/

Git files

.git/ .gitignore

OS files

.DS_Store Thumbs.db

Logs

*. log logs/

Test files

tests/ test *. py

Step 2: Why .dockerignore is Important

· Prevents unnecessary files from being copied · Reduces image size · Improves build speed Part 3: Building Docker Images

Step 1: Basic Build Command

Build image from Dockerfile

docker build -t my-flask-app .

Check built images

docker images

Step 2: Tagging Images

Tag with version number

docker build -t my-flask-app:1.0 .

Tag with multiple tags

docker build -t my-flask-app:latest -t my-flask-app:1.0 .

Tag with custom registry

docker build -t my-flask-app:1.0 .

Tag with multiple tags

docker build -t my-flask-app:latest -t my-flask-app:1.0 .

Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

Step 3: View Image Details

List all images

```
docker images
```

Show image history

```
docker history my-flask-app
```

Inspect image details

```
docker inspect my-flask-app
```

 Part 4: Running Containers

Step 1: Run Container

Run container with port mapping

```
docker run -d -p 5000:5000 -- name flask-container my-flask-app
```

Test the application

```
curl http://localhost:5000
```

View running containers

```
docker ps
```

View container logs

```
docker logs flask-container
```

Step 2: Manage Containers

Stop container

```
docker stop flask-container
```

Start stopped container

```
docker start flask-container
```

Remove container

```
docker rm flask-container
```

Remove container forcefully

```
docker rm -f flask-container
```

Step 1: Why Multi-stage Builds?

· Smaller final image size · Better security (remove build tools) · Separate build and runtime environments

Step 2: Simple Multi-stage Dockerfile

Dockerfile.multistage

STAGE 1: Builder stage

```
FROM python: 3.9-slim AS builder
```

STAGE 1: Builder stage

```
FROM python:3.9-slim AS builder
```

```
WORKDIR /app
```

Copy requirements

```
COPY requirements.txt .
```

Install dependencies in virtual environment

```
RUN python -m venv /opt/venv ENV PATH="/opt/venv/bin:$PATH" RUN pip install --no-cache-dir -r requirements.txt
```

STAGE 2: Runtime stage

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

Copy virtual environment from builder

```
COPY --from=builder /opt/venv /opt/venv ENV PATH="/opt/venv/bin:$PATH"
```

Copy application code

```
COPY app.py .
```

Create non-root user

```
RUN useradd -m -u 1000 appuser USER appuser
```

Expose port

```
EXPOSE 5000
```

Run application

```
CMD ["python", "app.py"] Step 3: Build and Compare
```

Build regular image

```
docker build -t flask-regular .
```

Build multi-stage image

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

Compare sizes

```
docker images | grep flask-
```

Expected output:

flask-regular

flask-multistage

~250MB ~150MB (40% smaller!) Part 6: Publishing to Docker Hub

Step 1: Prepare for Publishing

Login to Docker Hub

`docker login`

Tag image for Docker Hub

`docker tag my-flask-app:latest username/my-flask-app:1.0` `docker tag my-flask-app:latest username/my-flask-app:latest`

Push to Docker Hub

`docker push username/my-flask-app:1.0` `docker push username/my-flask-app:latest`

Step 2: Pull and Run from Docker Hub

Pull from Docker Hub (on another machine)

`docker pull username/my-flask-app:latest`

Run the pulled image

`docker run -d -p 5000:5000 username/my-flask-app:latest`

Part 7: Node.js Example (Quick Version)

Step 1: Node.js Application

`mkdir my-node-app` `cd my-node-app`

`app.js` `const express = require('express'); const app = express(); const port = 3000;`

`app.get('/', (req, res) => { res.send('Hello from Node.js Docker!'); });`

`app.get('/health', (req, res) => { res.json({ status: 'healthy' }); });`

`app.listen(port, () => { console.log(Server running on port ${port}); });`

`package.json` `{ "name": "node-docker-app", "version": "1.0.0", "main": "app.js", "dependencies": { "express": "^4.18.2" } }` Step 2: Node.js Dockerfile

FROM node: 18-alpine

WORKDIR /app

COPY package *.json . / RUN npm install -- only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

Step 3: Build and Run

Build image

docker build -t my-node-app .

Run container

docker run -d -p 3000:3000 --name node-container my-node-app

Test

curl http://localhost:3000



Containerization and DevOps

Experiment 5: Docker – Volumes, Environment Variables, Monitoring & Networks

Objective

To understand and implement core Docker concepts including volumes, environment variables, monitoring, and networking, and to build a real-world multi-container application.

◆ Part 1: Docker Volumes – Persistent Data Storage

Lab 1: Understanding Data Persistence

Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash
```

Inside container:

```
echo "Hello World" > /data/message.txt
cat /data/message.txt
exit
```

Restart container:

```
docker start test-container
docker exec test-container cat /data/message.txt
```

 Data is lost after restart.



Screenshot:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\HARSHITA> wsl
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -it --name test-container ubuntu /bin/bash
docker: Error response from daemon: Conflict. The container name "/test-container" is already in use by container "525047d170366b07c0abd5a630ab695439b17c617c74ecc2a7c137b5e3fe69a". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -it --name test-container1 ubuntu /bin/bash
root@fb27dba59366:/# mkdir /data
echo "Hello World" > /data/message.txt
cat /data/message.txt
Hello World
root@fb27dba59366:/# exit
exit
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker start test-container
test-container
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d -v /app/data --name web1 nginx
0a54d2e5d77d7904ed3f7055775eaa971efc06628ff88e51ee85958e5f7c3dc1
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume ls
DRIVER      VOLUME NAME
local       0ea9a3257976d2ced76b6eb84d7fe26ffe9ebf76d78c4849052995c71dc5f66f
local       6cea19db70bf95b7dd4ee18368e3384dc9f3652fe30a3b21225eac37afd5321d
local       91f69b9a7c4ab71147613c204f2baea2dbefa10396cc89c18cb7e9f9dce1acd5
local       495d7f351474c1b10a6455150567be5eb0f36b9b8c0bd39193d922d1639a24a9
local       96718d1a0c92822b8db0f6d1f3eac16ed360e9fc91511d8de1e45057d07fa0e1
local       2724621d53b973aa09f027af3c775df332b7c4b2a283dba38092fa26de69139e
local       a6ff8e473f231fdf708ce1b7cf858bc1a93931635dec57d89f0a4b593ef5630
local       docker-fastapi-postgres_postgres_data
local       e233334414384774582b4f6b3cb91e2aef7c40c8b05140d23cd019ca067f946b
local       f8c318efb7949565cecc9f6c0a48413328a824ba5c773b1c545fd3bd07eb8129
local       k3d-mycluster1-images
```



Lab 2: Volume Types

1. Anonymous Volume

```
docker run -d -v /app/data --name web1 nginx
```



Screenshot:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\HARSHITA> wsl
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -it --name test-container ubuntu /bin/bash
docker: Error response from daemon: Conflict. The container name "/test-container" is already in use by container "525047d170366b07c0abd5a630ab695439b17c617c74ecc2a7c137b5e3fe69a". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -it --name test-container1 ubuntu /bin/bash
root@fb27dba59366:/# mkdir /data
echo "Hello World" > /data/message.txt
cat /data/message.txt
Hello World
root@fb27dba59366:/# exit
exit
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker start test-container
test-container
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d -v /app/data --name web1 nginx
0a54d2e5d77d7904ed3f7055775eaa971efc06628ff88e51ee85958e5f7c3dc1
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume ls
DRIVER      VOLUME NAME
local       0ea9a3257976d2ced76b6eb84d7fe26ffe9ebf76d78c4849052995c71dc5f66f
local       6cea19db70bf95b7dd4ee18368e3384dc9f3652fe30a3b21225eac37afd5321d
local       91f69b9a7c4ab71147613c204f2baea2dbefa10396cc89c18cb7e9f9dce1acd5
local       495d7f351474c1b10a6455150567be5eb0f36b9b8c0bd39193d922d1639a24a9
local       96718d1a0c92822b8db0f6d1f3eac16ed360e9fc91511d8de1e45057d07fa0e1
local       2724621d53b973aa09f027af3c775df332b7c4b2a283dba38092fa26de69139e
local       a6ff8e473f231fdf708ce1b7cf858bc1a93931635dec57d89f0a4b593ef5630
local       docker-fastapi-postgres_postgres_data
local       e233334414384774582b4f6b3cb91e2aef7c40c8b05140d23cd019ca067f946b
local       f8c318efb7949565cecc9f6c0a48413328a824ba5c773b1c545fd3bd07eb8129
local       k3d-mycluster1-images
```

2. Named Volume

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
```

 Screenshot:

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume create mydata
mydata
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d -v mydata:/app/data --name web2 nginx
af0f8eafa5bbb32bcf8e2cca2494bc9496fd3b5d99e0055894d3095b9f76809f
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume ls
DRIVER      VOLUME NAME
local       0ea9a3257976d2ced76b6eb84d7fe26ffe9ebf76d78c4849052995c71dc5f66f
local       6cea19db70bf95b7dd4ee18368e3384dc9f3652fe30a3b21225eac37afd5321d
local       91f69b9a7c4ab71147613c204f2baea2dbefa10396cc89c18cb7e9f9dce1acd5
local       495d7f351474c1b10a6455150567be5eb0f36b9b8c0bd39193d922d1639a24a9
local       96718d1a0c92822b8db0f6d1f3eac16ed360e9fc91511d8de1e45057d07fa0e1
local       2724621d53b973aa09f027af3c775df332b7c4b2a283dba38092fa26de69139e
local       a6ff8e473f231fdf708ce1b7cf858bc1a93931635decd57d89f0a4b593ef5630
local       docker-fastapi-postgres_postgres_data
local       e233334414384774582b4f6b3cb91e2aef7c40c8b05140d23cd019ca067f946b
local       f8c318efb7949565cecc9f6c0a48413328a824ba5c773b1c545fd3bd07eb8129
local       k3d-mycluster1-images
local       k3d-mycluster-images
local       mydata
```

3. Bind Mount

```
docker run -d -v /mnt/c/Users/HARSHITA/myapp-data:/app/data --name web3 nginx
```

 Screenshot:

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ mkdir -p /mnt/c/Users/HARSHITA/myapp-data
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d -v /mnt/c/Users/HARSHITA/myapp-data:/app/data --name web3 nginx
569173b6f29aaf8015c27f4fb5a3ad7a638c1fa1d8a75f849f35a4d6ad00be4a
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ echo "From Host" > /mnt/c/Users/HARSHITA/myapp-data/host-file.txt
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker exec web3 cat /app/data/host-file.txt
From Host
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
139627f60883: Pull complete
caa784df8611: Pull complete
fe997c72f3e9: Pull complete
38fe0e0986de: Pull complete
bb5107df7baa: Pull complete
62f1cc0a64ca: Pull complete
4f7cfd23b111: Pull complete
c689d1836928: Pull complete
a0db31010df3: Pull complete
09b552d72653: Pull complete
b723faee7585: Pull complete
Digest: sha256:d0304ed9fdb64a3f6c7ad11a5fb4f13abfc10e6dfa3f288d652e7320c34df7f9
Status: Downloaded newer image for mysql:8.0
eb831a3d6b9501e6c8fc2414c7678e14b7988bc169f5554a064b81a7522b7797
```

Lab 3: Volume Commands

```
docker volume ls
docker volume create app-volume
docker volume inspect app-volume
docker volume prune
docker volume rm app-volume
docker cp local-file.txt web3:/app/data/
```

 Screenshot:

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume create app-volume
app-volume
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-23T09:57:13Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume rm app-volume
app-volume
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  nginx
b45d23c1a0ec44fe9631c0da89fe438f215bad4b24a9e36462283e3618931c3c
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker exec app1 printenv DATABASE_URL
postgres://user:pass@db:5432/mydb
true
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
```

◆ Part 2: Environment Variables

Lab 1: Using -e Flag

```
docker run -d \
  --name app1 \
  -e DEBUG=true \
  nginx
```

Lab 2: Using .env File

```
docker run -d --env-file .env --name app2 nginx
```


 Screenshot:

```
total test
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume create app-volume
app-volume
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-23T09:57:13Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker volume rm app-volume
app-volume
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  nginx
b45d23c1a0ec44fe9631c0da89fe438f215bad4b24a9e36462283e3618931c3c
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker exec app1 printenv DATABASE_URL
postgres://user:pass@db:5432/mydb
true
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
```

 Lab 3: Flask App with Environment Variables

```
docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST=prod-db \
  flask-app
```

 Screenshot:

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker run -d \
  --name flask-container \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e PORT="5000" \
  flask-app
```

NAME	CPU %	MEM USAGE / LIMIT	NET I/O
flask-container	--	-- / --	--
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
b81969965970	flask-container	0.11%	43.45MiB / 5.7GiB
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
b81969965970	flask-container	0.11%	43.45MiB / 5.7GiB
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
b81969965970	flask-container	0.29%	43.45MiB / 5.7GiB
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT
5180a52a1d27	test-nginx	0.00%	0B / 0B
b81969965970	flask-container	0.29%	43.45MiB / 5.7GiB

◆ Part 3: Docker Monitoring

Lab 1: docker stats

```
docker stats
```

 Screenshot:

```
harshita@DESKTOP-FM  x  harshita@DESKTOP-FMI  x  harshita@DESKTOP-FMI  x  harshita@DESKTOP-FMI  x  harshita@DESKTOP-FMI  x  +  -  X
4c6c80f5688f      boring_pascal      --      -- / --      --      --      --
5ad3f18b1bc4      gallant_mcnulty    --      -- / --      --      --      --
f76031f996a9      bold_joliot        --      -- / --      --      --      --

harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker top container-name
Error response from daemon: No such container: container-name
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker top test-nginx
Error response from daemon: container 5180a52ald27c3e6a6da4c47139528a645035c93a08e67aa5c30c48f325cae90 is not running
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker top test-nginx -ef
Error response from daemon: container 5180a52ald27c3e6a6da4c47139528a645035c93a08e67aa5c30c48f325cae90 is not running
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ ps aux | grep docker
root      716   0.0   0.4 1247368 24672 pts/1    Ssl+  09:46   0:02 /mnt/wsl/docker-desktop/docker-desktop-user-distro proxy --distro-na
me Ubuntu --docker-desktop-root /mnt/wsl/docker-desktop C:\Program Files\Docker\Docker\resources
harshita  5311  0.0   0.0   4088   1920 pts/5    S+   10:51   0:00 grep --color=auto docker
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker logs test-nginx
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker logs -f test-nginx
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker logs --tail 100 test-nginx
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker logs -t test-nginx
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker logs --since 2024-01-15 test-nginx
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker logs -f --tail 50 -t test-nginx
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect test-nginx
```

Lab 2: docker top

```
docker top flask-app
```

Lab 3: docker logs

```
docker logs -f flask-app
```

Lab 4: docker inspect

```
docker inspect flask-app
```

 Screenshot:

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.State.Status}}' test-nginx
created
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.NetworkSettings.IPAddress}}' test-nginx
10.99b63d
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.Config.Env}}' test-nginx
[PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin NGINX_VERSION=1.29.5 NJS_VERSION=0.9.5 NJS_RELEASE=1~trixie ACME_V
ERSION=0.3.1 PKG_RELEASE=1~trixie DYNPKG_RELEASE=1~trixie]
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.HostConfig.Memory}}' test-nginx
0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.HostConfig.NanoCpus}}' test-nginx
0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker events

2026-04-23T10:53:37.216784922Z container exec_create: /bin/sh -c curl -f http://localhost:8000/health || exit 1 d7939e74049204e49e37c
4fd6a0c1e0f0ede5f3fd4856e597dbfdelf23ecb8e4 (com.docker.compose.config-hash=a5df23dbc9a888f370fd1f74b8e5095818c1d8e42f69e4c6ac86693b9
109b63d, com.docker.compose.container-number=1, com.docker.compose.depends_on=database:service_started:false, com.docker.compose.imag
e=sha256:b256f71fc62fe8946f1dc7b7f16106bee70ee6a3705bd7077bc4331c70d2ae26, com.docker.compose.oneoff=False, com.docker.compose.projec
t=docker-fastapi-postgres, com.docker.compose.project.config_files=E:\docker-fastapi-postgres\docker-compose.yml, com.docker.compose.
```

 Lab 5: docker events

docker events

 Screenshot:

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.State.Status}}' test-nginx
created
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.NetworkSettings.IPAddress}}' test-nginx
10.99b63d
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.Config.Env}}' test-nginx
[PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin NGINX_VERSION=1.29.5 NJS_VERSION=0.9.5 NJS_RELEASE=1~trixie ACME_V
ERSION=0.3.1 PKG_RELEASE=1~trixie DYNPKG_RELEASE=1~trixie]
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.HostConfig.Memory}}' test-nginx
0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker inspect --format='{{.HostConfig.NanoCpus}}' test-nginx
0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker events

2026-04-23T10:53:37.216784922Z container exec_create: /bin/sh -c curl -f http://localhost:8000/health || exit 1 d7939e74049204e49e37c
4fd6a0c1e0f0ede5f3fd4856e597dbfdelf23ecb8e4 (com.docker.compose.config-hash=a5df23dbc9a888f370fd1f74b8e5095818c1d8e42f69e4c6ac86693b9
109b63d, com.docker.compose.container-number=1, com.docker.compose.depends_on=database:service_started:false, com.docker.compose.imag
e=sha256:b256f71fc62fe8946f1dc7b7f16106bee70ee6a3705bd7077bc4331c70d2ae26, com.docker.compose.oneoff=False, com.docker.compose.projec
t=docker-fastapi-postgres, com.docker.compose.project.config_files=E:\docker-fastapi-postgres\docker-compose.yml, com.docker.compose.

^Charshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker events --filter 'event=die'
^Charshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app# Since specific time
docker events --since '2024-01-15'

# Format output
docker events --format '{{.Type}} {{.Action}} {{.Actor.Attributes.name}}'
2026-04-23T10:20:45.940103916Z container die de83b39cb37516feaaeb33abf90a445593028a324e3f41e016ad4c372a6c21d9 (desktop.docker.io/port
s.scheme=v2, desktop.docker.io/wsl-distro=Ubuntu, execDuration=78, exitCode=0, image=env-image, name=env-container)
2026-04-23T10:21:11.560741417Z container exec_create: /bin/sh -c curl -f http://localhost:8000/health || exit 1 d7939e74049204e49e37c
4fd6a0c1e0f0ede5f3fd4856e597dbfdelf23ecb8e4 (com.docker.compose.config-hash=a5df23dbc9a888f370fd1f74b8e5095818c1d8e42f69e4c6ac86693b9
109b63d, com.docker.compose.container-number=1, com.docker.compose.depends_on=database:service_started:false, com.docker.compose.imag
e=sha256:b256f71fc62fe8946f1dc7b7f16106bee70ee6a3705bd7077bc4331c70d2ae26, com.docker.compose.oneoff=False, com.docker.compose.projec
t=docker-fastapi-postgres, com.docker.compose.project.config_files=E:\docker-fastapi-postgres\docker-compose.yml, com.docker.compose.
```

◆ Part 4: Docker Networks

 Lab 1: List Networks

docker network ls

 Screenshot:

```

^Charshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
e19c8ec5362a        bridge             bridge              local
56349ea28a3e        docker-fastapi-postgres_default bridge             local
f6e89729a5b6        docker_gwbridge     bridge             local
f6f0bd6e9b8a        host               host               local
cmwilutsy28e        ingress            overlay            swarm
f57b40b966f0        k3d-mycluster       bridge             local
49e5b02c1d08        k3d-mycluster1      bridge             local
e84513e1f08e        mymacvlan           macvlan            local
658f767f6256        none               null               local
58e68c8efcb5        sonarqube-project_sonarqube-lab bridge             local
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker network create my-network
890e19d953102db5d59098a498340c351badf0d580c28744893cee0d8d31deb0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker network inspect my-network
[
  {
    "Name": "my-network",
    "Id": "890e19d953102db5d59098a498340c351badf0d580c28744893cee0d8d31deb0",
    "Created": "2026-04-23T11:10:50.431812431Z",
    "Scope": "local",
  }
]

```

 Lab 2: Bridge Network

```

docker network create my-network
docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx

```

 Screenshot:

```

harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web1" is already in use by container "0a54d2e5d77d7904ed3f7055775eaa971efc06628ff88e51ee85958e5f7c3dc1". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
docker: Error response from daemon: Conflict. The container name "/web2" is already in use by container "af0f8eafa5bbb32bcf8e2cca2494bc9496fd3b5d99e0055894d3095b9f76809f". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker run -d --name web3 --network my-network nginx
docker run -d --name web4 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web3" is already in use by container "569173b6f29aaf8015c27f4fb5a3ad7a638c1fald8a75f849f35a4d6ad00be4a". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

```

 Lab 3: Network Commands

```

docker network inspect my-network
docker network connect my-network web1
docker network disconnect my-network web1

```

 Screenshot:

```

harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web1" is already in use by container "0a54d2e5d77d7904ed3f7055775eaa971efc06628ff88e51ee85958e5f7c3dc1". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
docker: Error response from daemon: Conflict. The container name "/web2" is already in use by container "af0f8eafa5bbb32bcf8e2cca2494bc9496fd3b5d99e0055894d3095b9f76809f". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/flask-app$ docker run -d --name web3 --network my-network nginx
docker run -d --name web4 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web3" is already in use by container "569173b6f29aaf8015c27f4fb5a3ad7a638c1fald8a75f849f35a4d6ad00be4a". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

```

◆ Part 5: Real-World Multi-Container Application



Architecture

- Flask App → Port 5000
- PostgreSQL → Port 5432
- Redis → Port 6379
- Custom Network



Implementation

```
docker network create myapp-network
```

```
docker run -d --name postgres --network myapp-network -e
POSTGRES_PASSWORD=mysecretpassword postgres:15
```

```
docker run -d --name redis --network myapp-network redis:7-alpine
```

```
docker run -d \
  --name flask-app \
  --network myapp-network \
  -p 5000:5000 \
  -e
DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" \
  flask-app
```

📸 Screenshot:

```
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker rm -f flask-app
flask-app
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker run -d \
  --name flask-app \
  --network myapp-network \
  -p 5000:5000 \
  -v $(pwd):/app \
  -v app-logs:/var/log/app \
  -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" \
  --env-file .env.production \
  flask-app
5208dc297532433bd74483fa28b127f1c8e063bde0a99ab56afce227e188ecdb
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint
flask-app (6d65fa9d57498feacf790b6b49e04c5df495755a4a51c576fc1c14858cf40d90): Bind for 0.0.0.0:5000 failed: port is already allocat
ed

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker run -d \
  --name flask-app \
  --network myapp-network \
  -p 5001:5000 \
  -v $(pwd):/app \
  -v app-logs:/var/log/app \
  -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
```

```

67ed2a107425ab09ed4018f58fd4a579660a37f4887895c5cc9b5bb9d5879724
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ curl http://localhost:5000
{"database": "postgres://postgres:mysecretpassword@postgres:5432/mydatabase", "debug": "false", "redis": "redis://redis:6379"}
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker ps
CONTAINER ID   IMAGE          NAMES      COMMAND                  CREATED        STATUS        PORTS
67ed2a107425   flask-app     flask-app  "python app.py"         28 seconds ago Up 27 seconds 0.0.0.0:5000-
>5000/tcp, [::]:5000->5000/tcp
bb7e3e02fa5d   redis:7-alpine  redis      "docker-entrypoint.s..." 4 minutes ago Up 4 minutes 6379/tcp
470567fe8bb0   postgres:15   postgres  "docker-entrypoint.s..." 5 minutes ago Up 4 minutes 5432/tcp
5703ce05cca1   postgres:15   postgres-db "docker-entrypoint.s..." 12 minutes ago Up 12 minutes 5432/tcp
291ded8544e6   alpine        isolated-app "sleep 3600"            15 minutes ago Up 15 minutes
bf6d98ca9b8a   nginx         host-app   "/docker-entrypoint...." 15 minutes ago Up 15 minutes

CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
470567fe8bb0   postgres  --        -- / --              --         --         --           --
bb7e3e02fa5d   redis     --        -- / --              --         --         --           --
67ed2a107425   flask-app --        -- / --              --         --         --           --

^C
got 3 SIGTERM/SIGINTs, forcefully exiting

harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker exec flask-app ping -c 2 postgres
docker exec flask-app ping -c 2 redis
PING postgres (172.24.0.2) 56(84) bytes of data.
64 bytes from postgres.myapp-network (172.24.0.2): icmp_seq=1 ttl=64 time=0.072 ms
64 bytes from postgres.myapp-network (172.24.0.2): icmp_seq=2 ttl=64 time=0.059 ms

--- postgres ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.059/0.065/0.072/0.006 ms
PING redis (172.24.0.3) 56(84) bytes of data.
64 bytes from redis.myapp-network (172.24.0.3): icmp_seq=1 ttl=64 time=0.175 ms
64 bytes from redis.myapp-network (172.24.0.3): icmp_seq=2 ttl=64 time=0.110 ms

--- redis ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.110/0.142/0.175/0.032 ms
harshita@DESKTOP-FMBUDGE:~/my-flask-app$ docker network inspect myapp-network
[
  {
    "Name": "myapp-network",
    "Id": "dbcfcc2cc92d22e5af318530fd3e061803e0b42e86f0a598e28539f5a848f75",
    "Created": "2026-04-23T11:23:25.279261813Z",
    "Scope": "local",
    "Driver": "bridge"
  }
]

```

Verification

```

docker ps
docker stats
docker logs -f flask-app
docker network inspect myapp-network

```

Result

Successfully implemented Docker concepts including volumes, environment variables, monitoring, and networking, and deployed a real-world multi-container application.

Key Takeaways

- Docker volumes ensure data persistence
 - Environment variables provide flexible configuration
 - Monitoring tools help debug containers
 - Networks enable container communication
 - Multi-container applications simulate real-world systems
-

Experiment 6 A

Comparison of Docker Run and Docker Compose

PART B - PRACTICAL TASK

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

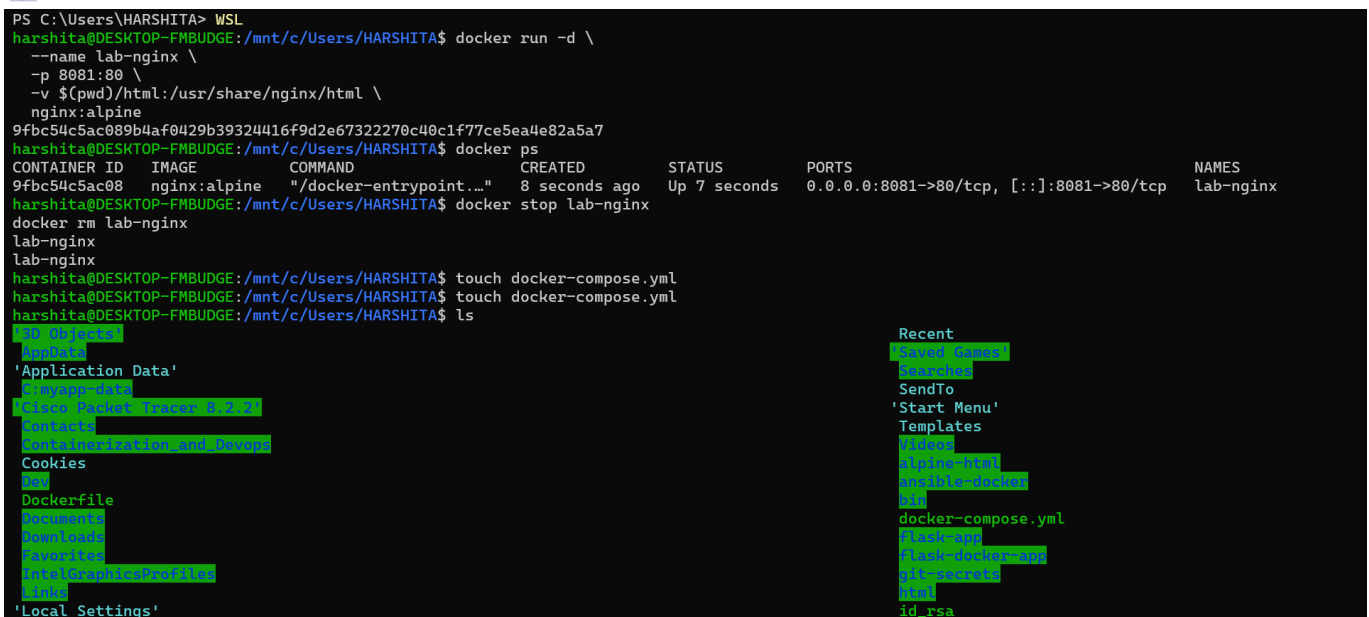
Execute

```
docker run -d \  
  --name lab-nginx \  
  -p 8081:80 \  
  -v $(pwd)/html:/usr/share/nginx/html \  
  nginx:alpine
```

Verify

```
docker ps
```

Screenshot:



```
PS C:\Users\HARSHITA> WSL  
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \  
  --name lab-nginx \  
  -p 8081:80 \  
  -v $(pwd)/html:/usr/share/nginx/html \  
  nginx:alpine  
9fbc54c5ac089b4af0429b39324416f9d2e67322270c40c1f77ce5ea4e82a5a7  
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES  
9fbc54c5ac08   nginx:alpine  "/docker-entrypoint..."  8 seconds ago  Up 7 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  lab-nginx  
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker stop lab-nginx  
lab-nginx  
lab-nginx  
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ touch docker-compose.yml  
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ touch docker-compose.yml  
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ ls  
..  .objects  Recent  
..data  Recent Games  
'Application Data'  Searches  
  .wwwup-0411  SendTo  
  .code  'Start Menu'  
  .code  Templates  
  .code  Videos  
  .code  alpine-nginx  
  .code  ansible-docker  
  .code  .id  
  .code  docker-compose.yml  
  .code  flask-app  
  .code  flask-docker-app  
  .code  git-secrets  
  .code  .id_rsa  
  .code  id_rsa  
Documents  
Downloads  
Favorites  
IntelGraphicsProfile  
links  
'Local Settings'
```

Access

http://localhost:8081

Stop & Remove

```
docker stop lab-nginx
docker rm lab-nginx
```

Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml`

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Run

```
docker compose up -d
```

Verify

```
docker compose ps
```

 Screenshot:

```

PrintHood
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
✔Network harshita_default Created 0.1s
✔Container lab-nginx Started 1.1s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose ps
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid pot
l confusion
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
lab-nginx nginx:alpine "/docker-entrypoint..." nginx 7 seconds ago Up 6 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose down
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
✔Container lab-nginx Removed 10.7s
✔Network harshita_default Removed 0.4s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker network create wp-net
a85f7197bcb387c3fcf93a2900c9f996711ad4423a7009f1fc81f30d710d92c6
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
--name mysql \
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
Unable to find image 'mysql:5.7' locally
docker: error getting credentials - err: exit status 1, out: ``

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano ~/.docker/config.json
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker pull mysql:5.7
5.7: Pulling from library/mysql
1c56c3d4ce74: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete

```

Stop

```
docker compose down
```

Task 2: Multi-Container Application

Objective

Deploy WordPress with MySQL using:

- Docker Run (manual way)
- Docker Compose (structured way)

A. Using Docker Run

Step 1: Create Network

```
docker network create wp-net
```

Step 2: Run MySQL

```
docker run -d \
--name mysql \
```

```
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
```

Step 3: Run WordPress

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

Test

<http://localhost:8082>

 Screenshot:

```
PrintHood
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
  ✓ Network harshita_default Created 0.1s
  ✓ Container lab-nginx Started 1.1s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose ps
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid pot
l confusion
NAME          IMAGE          COMMAND                  SERVICE    CREATED      STATUS      PORTS
lab-nginx     nginx:alpine   "/docker-entrypoint..." nginx       7 seconds ago Up 6 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose down
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
  ✓ Container lab-nginx Removed 10.7s
  ✓ Network harshita_default Removed 0.4s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker network create wp-net
a85f7197bcb387c3fcf93a2900c9f996711ad4423a7009f1fc81f30d710d92c6
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
Unable to find image 'mysql:5.7' locally
docker: error getting credentials - err: exit status 1, out: ``

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano ~/.docker/config.json
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker pull mysql:5.7
5.7: Pulling from library/mysql
1c56c3d4ce74: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete
```

```

docker.io/library/mysql:5.7
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
c6f14eb8dad6c699d3970d3b41a55f65b1349054744c4c507460249338db9afe0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
a5f587e05427: Pull complete
899cbc2fdb85: Pull complete
f7417fc69aeb: Pull complete
cac4a54c06df: Pull complete
bd5f94b94043: Pull complete
a923b3cdaa21: Pull complete
2b5354e1530f829a0f8715426f9c95e691fa1fba609dd525786a27f7783a4b67
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS
2b5354e1530f   wordpress:latest  "docker-entrypoint.s..."  28 seconds ago  Up 24 seconds      0.0.0.0:8082->80/tcp
c6f14eb8dad6   mysql:5.7        "docker-entrypoint.s..."  2 minutes ago   Up 2 minutes       3306/tcp
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker stop wordpress mysql
docker rm wordpress mysql
docker network rm wp-net
wordpress
mysql
wordpress
mysql
wp-net
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored
1 confusion

```

B. Using Docker Compose

Create ``bash

docker compose down

```
---
```

```
## Task 2: Multi-Container Application
```

```
### Objective
```

```
Deploy WordPress with MySQL using:
```

- * Docker Run (manual way)
- * Docker Compose (structured way)

```
---

### A. Using Docker Run

#### Step 1: Create Network

```bash
docker network create wp-net
```

## Step 2: Run MySQL

```
docker run -d \
 --name mysql \
 --network wp-net \
 -e MYSQL_ROOT_PASSWORD=secret \
 -e MYSQL_DATABASE=wordpress \
 mysql:5.7
```

## Step 3: Run WordPress

```
docker run -d \
 --name wordpress \
 --network wp-net \
 -p 8082:80 \
 -e WORDPRESS_DB_HOST=mysql \
 -e WORDPRESS_DB_PASSWORD=secret \
 wordpress:latest
```

## Test

<http://localhost:8082>

## Screenshot:

```

PrintHood
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
 ✓ Network harshita_default Created 0.1s
 ✓ Container lab-nginx Started 1.1s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose ps
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pot
l confusion
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
lab-nginx nginx:alpine "/docker-entrypoint...." nginx 7 seconds ago Up 6 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose down
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pot
l confusion
[+] Running 2/2
 ✓ Container lab-nginx Removed 10.7s
 ✓ Network harshita_default Removed 0.4s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker network create wp-net
a85f7197bcb387c3fcf93a2900c9f996711ad4423a7009f1fc81f30d710d92c6
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
 --name mysql \
 --network wp-net \
 -e MYSQL_ROOT_PASSWORD=secret \
 -e MYSQL_DATABASE=wordpress \
 mysql:5.7
Unable to find image 'mysql:5.7' locally
docker: error getting credentials - err: exit status 1, out: ``

Run 'docker run --help' for more information
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano ~/.docker/config.json
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker pull mysql:5.7
5.7: Pulling from library/mysql
1c56c3d4ce74: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete

docker.io/library/mysql:5.7
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
 --name mysql \
 --network wp-net \
 -e MYSQL_ROOT_PASSWORD=secret \
 -e MYSQL_DATABASE=wordpress \
 mysql:5.7
c6f14eb8dad6c699d3970d3b41a55f65b1349054744c4c507460249338db9afe0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
 --name wordpress \
 --network wp-net \
 -p 8082:80 \
 -e WORDPRESS_DB_HOST=mysql \
 -e WORDPRESS_DB_PASSWORD=secret \
 wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
a5f587e05427: Pull complete
899cbc2fdb85: Pull complete
f7417fc69aeb: Pull complete
cac4a54c06df: Pull complete
bd5f94b94043: Pull complete
a923b3cdaa21: Pull complete
2b5354e1530f829a0f8715426f9c95e691fa1fba609dd525786a27f7783a4b67
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
2b5354e1530f wordpress:latest "docker-entrypoint.s..." 28 seconds ago Up 24 seconds 0.0.0.0:80
c6f14eb8dad6 mysql:5.7 "docker-entrypoint.s..." 2 minutes ago Up 2 minutes 3306/tcp,
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker stop wordpress mysql
docker rm wordpress mysql
docker network rm wp-net
wordpress
mysql
wordpress
mysql
wp-net
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute `version` is obsolete, it will be i
l confusion

```

## B. Using Docker Compose

### Create `docker-compose.yml`

```
version: '3.8'

services:
 mysql:
 image: mysql:5.7
 environment:
 MYSQL_ROOT_PASSWORD: secret
 MYSQL_DATABASE: wordpress
 volumes:
 - mysql_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
 ports:
 - "8082:80"
 environment:
 WORDPRESS_DB_HOST: mysql
 WORDPRESS_DB_PASSWORD: secret
 depends_on:
 - mysql

volumes:
 mysql_data:
```

### Run

```
docker compose up -d
```

### Stop

```
docker compose down -v
```

## Screenshot:

```

✓Container webapp Started
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0001] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please
l confusion
[+] Running 4/4
✓backend Pulled
✓7ccd73948dde Pull complete
✓f3ba2250c524 Pull complete
✓91ff8760033c Pull complete
[+] Running 2/2
✓Network harshita_app-net Created
[+] Running 4/4hita_pgdata" Created
✓Network harshita_app-net Created
✓Volume "harshita_pgdata" Created
✓Container postgres-db Started
✓Container backend Started
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please
l confusion
WARN[0000] Found orphan containers ([webapp]) for this project. If you removed or renamed this service in your compos
mmand with the --remove-orphans flag to clean it up.
[+] Running 2/2
✓Container postgres-db Running
✓Container backend Started
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose ps
✓750a2200c024 Pull complete 0.8s
✓91ff8760033c Pull complete 2.8s
[+] Running 2/2
✓Network harshita_app-net Created 0.2s
[+] Running 4/4hita_pgdata" Created 0.0s
✓Network harshita_app-net Created 0.2s
✓Volume "harshita_pgdata" Created 0.0s
✓Container postgres-db Started 1.9s
✓Container backend Started 1.7s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose up -d
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potent
l confusion
WARN[0000] Found orphan containers ([webapp]) for this project. If you removed or renamed this service in your compose file, you can run this c
mmand with the --remove-orphans flag to clean it up.
[+] Running 2/2
✓Container postgres-db Running 0.0s
✓Container backend Started 0.5s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose ps
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potent
l confusion
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
postgres-db postgres:15 "docker-entrypoint.s..." postgres-db 21 seconds ago Up 20 seconds 5432/tcp
webapp node:18-alpine "docker-entrypoint.s..." webapp 31 minutes ago Restarting (0) 12 seconds ago
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker compose down -v
WARN[0000] /mnt/c/Users/HARSHITA/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potent
l confusion
[+] Running 4/4
✓Container backend Removed 0.1s
✓Container postgres-db Removed 0.6s
✓Volume harshita_pgdata Removed 0.2s
✓Network harshita_app-net Removed 0.5s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
--name limited-app \
-p 9000:9000 \

```

## PART C - CONVERSION & BUILD-BASED TASKS

### Task 3: Convert Docker Run to Docker Compose

#### Problem 1: Basic Web Application

##### Given Docker Run Command

```

docker run -d \
 --name webapp \
 -p 5000:5000 \
 -e APP_ENV=production \

```



```
-e DEBUG=false \
--restart unless-stopped \
node:18-alpine
```

## Docker Compose Equivalent

```
version: '3.8'

services:
 webapp:
 image: node:18-alpine
 container_name: webapp
 ports:
 - "5000:5000"
 environment:
 APP_ENV: production
 DEBUG: "false"
 restart: unless-stopped
```

## Run & Verify

```
docker compose up -d
docker compose ps
```

📷 Screenshot:

```
version: '3.8'

services:
 mysql:
 image: mysql:5.7
 environment:
 MYSQL_ROOT_PASSWORD: secret
 MYSQL_DATABASE: wordpress
 volumes:
 - mysql_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
 ports:
 - "8082:80"
 environment:
 WORDPRESS_DB_HOST: mysql
 WORDPRESS_DB_PASSWORD: secret
 depends_on:
 - mysql
```

```

volumes:
 mysql_data:
  ```

#### Run

```bash
docker compose up -d
```

#### Stop

```bash
docker compose down -v
```

📸 Screenshot:
![[WordPress Docker Compose](../screenshots/lab6/Screenshot%202026-04-25%20140325.png)]
![[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140354.png)]

---

# PART C - CONVERSION & BUILD-BASED TASKS

---

## Task 3: Convert Docker Run to Docker Compose

---

### Problem 1: Basic Web Application

#### Given Docker Run Command

```bash
docker run -d \
 --name webapp \
 -p 5000:5000 \
 -e APP_ENV=production \
 -e DEBUG=false \
 --restart unless-stopped \
 node:18-alpine
```

#### Docker Compose Equivalent

```yaml
version: '3.8'

services:
 webapp:

```

```

image: node:18-alpine
container_name: webapp
ports:
 - "5000:5000"
environment:
 APP_ENV: production
 DEBUG: "false"
restart: unless-stopped

```

#### Run & Verify

```

```bash
docker compose up -d
docker compose ps
```

```

 Screenshot:

```

`![WebApp Compose](../screenshots/lab6/Screenshot%202026-04-25%20140405.png)
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140412.png)

```

---

### Problem 2: Volume + Network Configuration

#### Docker Compose File

```

```yaml
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:

```

```

    - postgres-db
networks:
    - app-net

volumes:
    pgdata:

networks:
    app-net:
    ...

#### Run

```bash
docker compose up -d
```

#### Stop

```bash
docker compose down -v
```

📸 Screenshot:
`![Backend + DB](../screenshots/lab6/Screenshot%202026-04-25%20140422.png)`
`![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140429.png)`
`![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140238.png)`

---

## Task 4: Resource Limits Conversion

#### Docker Compose

```yaml
version: '3.8'

services:
 limited-app:
 image: nginx:alpine
 container_name: limited-app
 ports:
 - "9000:9000"
 restart: always
 deploy:
 resources:
 limits:
 cpus: "0.5"
 memory: 256M
 ...

Explanation

```

```
* ```yaml
version: '3.8'

services:
 limited-app:
 image: nginx:alpine
 container_name: limited-app
 ports:
 - "9000:9000"
 restart: always
 deploy:
 resources:
 limits:
 cpus: "0.5"
 memory: 256M
```
```

Explanation

- * `deploy` works only in Docker Swarm mode
- * In normal Compose → ignored
- * In Swarm → enforced

📸 Screenshot:

```
`![Resource Limits](../screenshots/lab6/Screenshot%202026-04-25%20213901.png)
![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20214338.png)
```

PART D - USING DOCKERFILE

Task 5: Replace Standard Image with Dockerfile

Step 1: Create `app.js`

```
```javascript
const http = require('http');

http.createServer((req, res) => {
 res.end("Docker Compose Build Lab");
}).listen(3000);
```
```

Step 2: Create Dockerfile

```
```dockerfile
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY app.js .
```

```
EXPOSE 3000
```

```
CMD ["node", "app.js"]
```

```
````
```

```
---
```

```
### Step 3: Create docker-compose.yml
```

```
```yaml
```

```
version: '3.8'
```

```
services:
```

```
 nodeapp:
```

```
 build:
```

```
 context: .
```

```
 dockerfile: Dockerfile
```

```
 container_name: custom-node-app
```

```
 ports:
```

```
 - "3000:3000"
```

```
````
```

```
---
```

```
#### Run
```

```
```bash
```

```
docker compose up --build -d
```

```
````
```

```
#### Verify
```

```
http://localhost:3000
```

```
📷 Screenshot:
```

```
`![Node App](../screenshots/lab6/Screenshot%202026-04-25%20214106.png)`
```

```
---
```

```
### Difference: image vs build
```

| Feature | image | build |
|---------------|------------|------------------|
| Source | Docker Hub | Local Dockerfile |
| Control | Low | High |
| Customization | No | Yes |

```
---
```

```
## Task 6: Multi-Stage Dockerfile
```

Requirements

- * Multi-stage build
- * Smaller image
- * Use Compose
- * Add environment variables
- * Add volume mount

Verify Image Size

```
```bash
```

```
docker images
```

```
```
```

📷 Screenshot:

![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140437.png)

Experiment 6 B

Multi-Container Application (WordPress + MySQL)

Objective

- * Deploy WordPress + MySQL
- * Understand networking & volumes
- * Learn scaling
- * Compare with Swarm

Architecture

```
```
```

User → WordPress → MySQL → Volume

```
```
```

Step 1: Create Directory

```
```bash
```

```
mkdir wp-compose-lab
```

```
cd wp-compose-lab
```

```
```
```

Step 2: Create docker-compose.yml

```
```yaml
```

```
version: '3.9'
```

```

services:
 db:
 image: mysql:5.7
 container_name: wordpress_db
 restart: always
 environment:
 MYSQL_ROOT_PASSWORD: rootpass
 MYSQL_DATABASE: wordpress
 MYSQL_USER: wpuser
 MYSQL_PASSWORD: wppass
 volumes:
 - db_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
 container_name: wordpress_app
 depends_on:
 - db
 ports:
 - "8080:80"
 restart: always
 environment:
 WORDPRESS_DB_HOST: db:3306
 WORDPRESS_DB_USER: wpuser
 WORDPRESS_DB_PASSWORD: wppass
 WORDPRESS_DB_NAME: wordpress
 volumes:
 - wp_data:/var/www/html

```

```

volumes:
 db_data:
 wp_data:
 ...

```

## Step 3: Start Application

```

```bash
docker-compose up -d
```

```

📷 Screenshot:

```

```bash
docker images
```

```

📷 Screenshot:

```

![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140437.png)

```

# Experiment 6 B

## Multi-Container Application (WordPress + MySQL)



---

## ## Objective

- \* Deploy WordPress + MySQL
- \* Understand networking & volumes
- \* Learn scaling
- \* Compare with Swarm

---

## ## Architecture

```

User → WordPress → MySQL → Volume
```

---

## ## Step 1: Create Directory

```
```bash
mkdir wp-compose-lab
cd wp-compose-lab
```
```

---

## ## Step 2: Create docker-compose.yml

```
```yaml
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
```

```
restart: always
environment:
  WORDPRESS_DB_HOST: db:3306
  WORDPRESS_DB_USER: wpuser
  WORDPRESS_DB_PASSWORD: wppass
  WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html
```

```
volumes:
  db_data:
  wp_data:
  ...
```

Step 3: Start Application

```
```bash
docker-compose up -d
```
```



Screenshot:

```
`![WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140442.png)`
```

Step 4: Verify

```
```bash
docker ps
```
```

Step 5: Access WordPress

<http://localhost:8080>



Screenshot:

```
`![WordPress Setup](../screenshots/lab6/Screenshot%202026-04-25%20214338.png)`
```

Step 6: Check Volumes

```
```bash
docker volume ls
```
```

Step 7: Stop

```
```bash
docker-compose down
```

---

## Scaling in Docker Compose

```bash
docker-compose up --scale wordpress=3
```

### Issues

* Port conflict
* No load balancing

### Solution

Use Nginx Reverse Proxy

---

## Docker Swarm

### Initialize

```bash
docker swarm init
```

### Deploy Stack

```bash
docker stack deploy -c docker-compose.yml wpstack
```

### Scale

```bash
docker service scale wpstack_wordpress=3
```

📸 Screenshot:
`![Swarm](../screenshots/lab6/Screenshot%202026-04-25%20214439.png)`
[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20140437.png)
[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20214707.png)
[WordPress Docker Run](../screenshots/lab6/Screenshot%202026-04-25%20214721.png)

---

## Benefits of Docker Swarm

* Load balancing
```

- * Auto-healing
- * Scaling
- * Rolling updates

Limitations

- * Less popular than Kubernetes
- * Limited ecosystem

Key Learning Outcomes

- * Multi-container apps need orchestration
- * Docker Compose is ideal for:
 - * Development
 - * Testing
 - * Learning

Experiment 7

CI/CD using Jenkins, GitHub and Docker Hub

Name: Harshita Chabaria

Subject: Containerization and DevOps

Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
 - Create a structured GitHub repository with application and Jenkinsfile
 - Build Docker images from source code
 - Securely store Docker Hub credentials in Jenkins
 - Automate build and push process using webhook triggers
 - Use same host (Docker) as Jenkins agent
-

Theory

1. What is Jenkins?

Jenkins is a web-based GUI automation server used to build, test, and deploy applications. It supports plugins (GitHub, Docker, etc.) and uses **Pipeline as Code (Jenkinsfile)**.

2. What is CI/CD?

- **Continuous Integration (CI):** Automatically builds and tests code after each commit
 - **Continuous Deployment (CD):** Automatically delivers built artifacts (Docker images)
-

3. Workflow Overview

```
Developer → GitHub → Webhook → Jenkins → Build → Docker Hub
```

4. Prerequisites

- Docker & Docker Compose installed
 - GitHub account
 - Docker Hub account
 - Basic Linux knowledge
-

Part A: GitHub Repository Setup

Project Structure

```
My-App/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
└── Jenkinsfile
```

Application Code

app.py

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hello from CI/CD Pipeline!"  
  
app.run(host="0.0.0.0", port=80)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.10-slim
```

```
WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80

CMD ["python", "app.py"]
```

Jenkinsfile

```
pipeline {
  agent any

  environment {
    IMAGE_NAME = "har4049/myapp"
  }

  stages {

    stage('Clone Source') {
      steps {
        git branch: 'main', url:
'https://github.com/harshitachabaria24/My-App'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh 'docker build -t $IMAGE_NAME:latest .'
      }
    }

    stage('Login to Docker Hub') {
      steps {
        withCredentials([string(credentialsId: 'dockerhub-token',
variable: 'DOCKER_TOKEN')]) {
          sh 'echo $DOCKER_TOKEN | docker login -u jasman1201 --
password-stdin'
        }
      }
    }

    stage('Push to Docker Hub') {
      steps {
        sh 'docker push $IMAGE_NAME:latest'
      }
    }
  }
}
```

Part B: Jenkins Setup using Docker

docker-compose.yml

```
version: '3.8'

services:
  jenkins:
```



```
image: jenkins/jenkins:lts
container_name: jenkins
restart: always
ports:
  - "8080:8080"
  - "50000:50000"
volumes:
  - jenkins_home:/var/jenkins_home
  - /var/run/docker.sock:/var/run/docker.sock
user: root

volumes:
  jenkins_home:
```

Steps

Run Jenkins

```
docker compose up -d
```

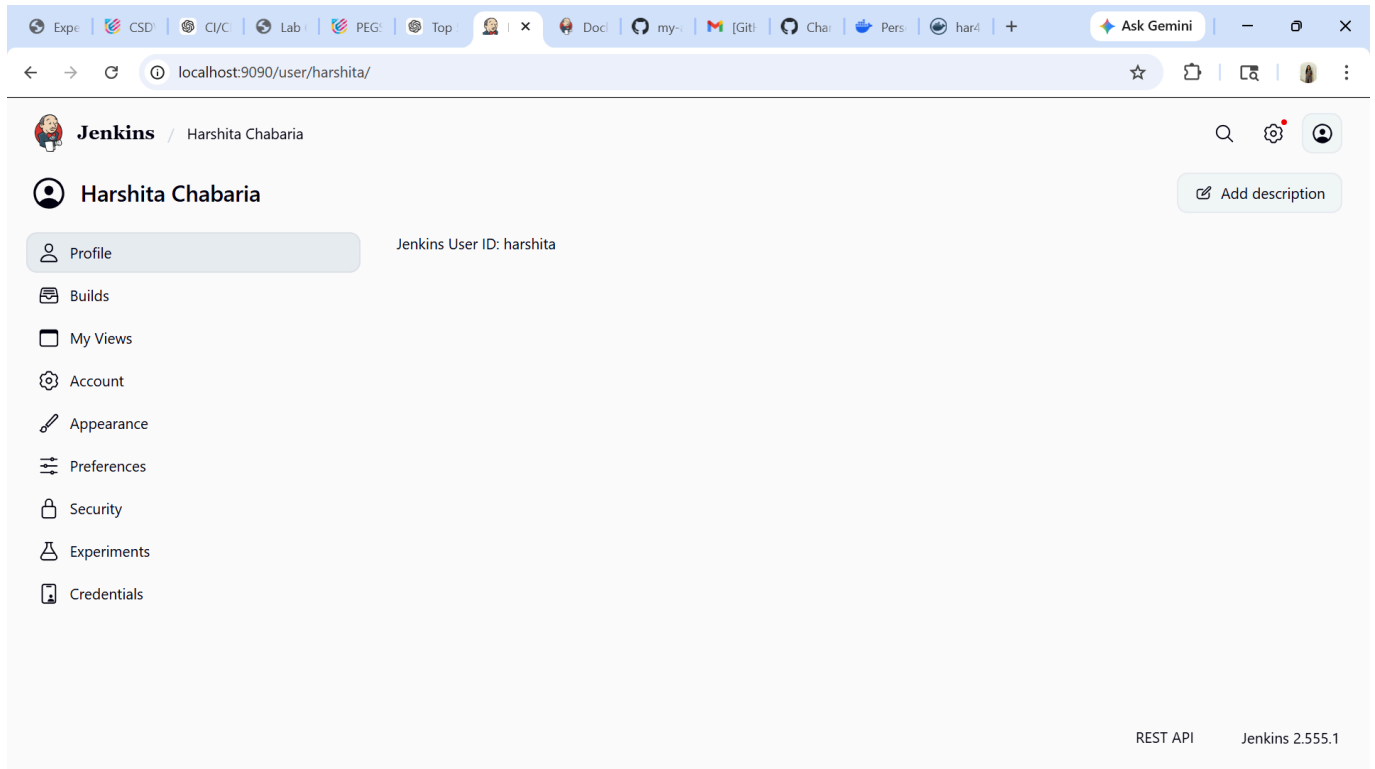


Screenshot: `

```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker run -d \
-p 9090:8080 \
-p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
jenkins/jenkins:lts
b0d0a7a02241de0d0c50ba4cfb6767c175851621d12d8a5feef04a7ba90f9d00
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ docker ps
```

Access Jenkins

<http://localhost:8080> ![Jenkins Started](../screenshots/lab7/Screenshot%202026-04-25%20225057.png)



Get Admin Password

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Install Plugins

- Click **Install Suggested Plugins**

Install Docker inside Jenkins

```
apt-get update && apt-get install -y docker.io
```

 Screenshot:

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Install Plugins

* Click Install Suggested Plugins

```

### Install Docker inside Jenkins

```
```bash id="dockerinstall"
apt-get update && apt-get install -y docker.io
```
```

 Screenshot:

`![Docker Installed](../screenshots/lab7/Screenshot%20(693).png)`

---

### Verify Docker

```
```bash id="dockerps"
docker ps
```
```

 Screenshot:

`![Docker PS](../screenshots/lab7/Screenshot%20(695).png)`

`![Console Output](../screenshots/lab7/Screenshot%20(694).png)`

---

# Part C: Jenkins Configuration

---

### Add Docker Hub Credentials

Path:

**\*\*Manage Jenkins → Credentials → Global → Add Credentials\*\***

 Screenshot:

`![Credentials](../screenshots/lab7/Screenshot%20(688).png)`

---

### Configure Pipeline Job

- \* New Item → Pipeline
- \* Pipeline script from SCM
- \* Git Repo URL
- \* Branch: main
- \* Script Path: Jenkinsfile

 Screenshot:

`![Pipeline Config](../screenshots/lab7/Screenshot%20(690).png)`

---

### Pipeline Execution

 Screenshot:

`![Console Output](../screenshots/lab7/Screenshot%20(687).png)`

```
`![Console Output](../screenshots/lab7/Screenshot%20(697).png)
```

```

```

## ## Execution Flow

| Stage              | Action                | Status    |
|--------------------|-----------------------|-----------|
| Clone Source       | Pull code from GitHub | ✓ Success |
| Build Docker Image | Build image           | ✓ Success |
| Login Docker Hub   | Authenticate          | ✓ Success |
| Push Image         | Push to Docker Hub    | ✓ Success |

```

```

## ## Role of Same Host Agent

Jenkins runs inside Docker with socket mounted:

```
```id="socket"
/var/run/docker.sock → /var/run/docker.sock
```
```

➡ This allows Jenkins to control host Docker directly

```

```

## # Observations

- \* Jenkins GUI simplifies CI/CD
- \* GitHub stores code + pipeline
- \* Docker ensures consistency
- \* Credentials stored securely
- \* Docker socket enables direct control

```

```

## # Result

Successfully implemented CI/CD pipeline where:

- \* Code + Jenkinsfile in GitHub
- \* Jenkins auto-builds on changes
- \* Docker image built
- \* Image pushed to Docker Hub securely

```

```

# Experiment 9

---

## Ansible

---

## Theory

---

### Problem Statement

Managing infrastructure manually across multiple servers leads to:

- Configuration drift
- Inconsistent environments
- Time-consuming repetitive tasks

Scaling from one server to hundreds becomes difficult using manual SSH-based administration.

---

### What is Ansible?

- Open-source automation tool for configuration management, deployment, and orchestration
  - Agentless architecture (uses SSH for Linux, WinRM for Windows)
  - Uses YAML-based playbooks
  - Industry-standard automation tool
- 

### How Ansible Solves the Problem

- **Agentless** → No installation on target machines
  - **Idempotent** → Same result every time
  - **Declarative** → Define desired state
  - **Push-based** → Control node sends instructions
- 

### Key Concepts

| Component     | Description                    |
|---------------|--------------------------------|
| Control Node  | Machine with Ansible installed |
| Managed Nodes | Target servers                 |
| Inventory     | List of servers                |
| Playbooks     | YAML automation files          |
| Tasks         | Individual actions             |
| Modules       | Built-in functions             |

---

## How Ansible Works

- Control node connects to managed nodes via SSH
- Executes modules through tasks
- Tasks combine into playbooks
- Inventory defines servers

---

## Benefits

- Free & open-source
- Easy to learn
- No agents required
- Reusable and modular

---

## Part A: Installation

### Install using pip

```
pip install ansible
ansible --version
```

 Screenshot: `

```
harshita@DESKTOP-FMBUDGE:~/ansible-docker$ sudo apt update
[sudo] password for harshita:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1620 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1919 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [345 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.1 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1685 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [259 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:14 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7356 B]
Get:15 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.9 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [11.0 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1182 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Fetched 8095 kB in 8s (965 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
47 packages can be upgraded. Run 'apt list --upgradable' to see them.
harshita@DESKTOP-FMBUDGE:~/ansible-docker$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

## Install using apt

```
sudo apt update -y
sudo apt install ansible -y
ansible --version
```

 Screenshot:

```
sudo apt update -y
sudo apt install ansible -y
ansible --version
```
```

 Screenshot:

```
`![Ansible Install Apt](../screenshots/lab9/Screenshot%202026-04-26%20012854.png)
```

```
## Test Installation
```

```
```bash id="pingtest"
ansible localhost -m ping
```
```

```
### Expected Output
```

```
```bash id="expected"
localhost | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
```
```

 Screenshot:

```
`![Ansible Ping](../screenshots/lab9/Screenshot%202026-04-26%20012904.png)
```

```
# Part B: Docker + SSH Setup
```

```
## Step 1: Create SSH Key Pair
```

```
```bash id="sshkey"
ssh-keygen -t rsa -b 4096
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```
```

 Screenshot:

```
```bash id="sshkey"
ssh-keygen -t rsa -b 4096
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```
```

 Screenshot:

```
`![SSH Key](../screenshots/lab9/Screenshot%202026-04-26%20012738.png)
```

Step 2: Create Dockerfile

```
```dockerfile id="dockerfile"
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

RUN mkdir -p /run/sshd && \
 echo 'root:password' | chpasswd && \
 sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/'
/etc/ssh/sshd_config && \
 sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/'
/etc/ssh/sshd_config && \
 sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/'
/etc/ssh/sshd_config

RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh

COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

RUN chmod 600 /root/.ssh/id_rsa && \
 chmod 644 /root/.ssh/authorized_keys

RUN sed -i 's@session\\s*required\\s*pam_loginuid.so@session optional
pam_loginuid.so@g' /etc/pam.d/sshd

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```
```

 Screenshot:

```
`![Dockerfile](../screenshots/lab9/Screenshot%202026-04-26%20012753.png)
```

Step 3: Build Image

```
```bash id="building"
```



```
docker build -t ubuntu-server .
```
```

📷 Screenshot:

```
````bash id="buildimg"  
docker build -t ubuntu-server .
```
```

📷 Screenshot:

```
`![Docker Build](../screenshots/lab9/Screenshot%202026-04-26%20012753.png)
```

Step 4: Run Container

```
````bash id="runcontainer"  
docker run -d -p 2222:22 --name ssh-test-server ubuntu-server
```
```

📷 Screenshot:

```
`![Docker Run](../screenshots/lab9/Screenshot%202026-04-26%20012812.png)
```

Step 5: Get Container IP

```
````bash id="getip"  
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-
test-server
```
```

📷 Screenshot:

```
````bash id="getip"  
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-
test-server
```
```

📷 Screenshot:

```
`![Container IP](../screenshots/lab9/Screenshot%202026-04-26%20012827.png)
```

Step 6: Test SSH

Password Login

```
````bash id="sshpass"  
ssh root@localhost -p 2222
```
```

Key-based Login

```
````bash id="sshkeylogin"  
ssh -i ~/.ssh/id_rsa root@localhost -p 2222
```

```

📷 Screenshot:

^[SSH Login](../screenshots/lab9/Screenshot%202026-04-26%20012827.png)

Cleanup

```
```bash id="cleanup"
docker stop ssh-test-server
docker rm ssh-test-server
```
```

📷 Screenshot:

```
```bash id="cleanup"
docker stop ssh-test-server
docker rm ssh-test-server
```
```

📷 Screenshot:

^[SSH Login](../screenshots/lab9/Screenshot%202026-04-26%20012904.png)

Part C: Ansible with Docker

Step 1: Start Multiple Containers

```
```bash id="multicontainer"
for i in {1..4}; do
 docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
done
```
```

📷 Screenshot:

^[Multiple Servers](../screenshots/lab9/Screenshot%202026-04-30%20224342.png)

Stop All Servers

```
```bash id="stopservers"
for i in {1..4}; do
 docker stop server${i}
done
```
```

Step 2: Create Inventory

```
```bash id="inventory"
```

```

echo "[servers]" > inventory.ini

for i in {1..4}; do
 docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
server${i} >> inventory.ini
done

cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

---

## Step 3: View Inventory

```bash id="viewinventory"
cat inventory.ini
```

---

## Step 4: Test Connectivity

```bash id="ansibleping"
ansible all -i inventory.ini -m ping
```

📸 Screenshot:

```bash id="stopservers"
for i in {1..4}; do
 docker stop server${i}
done
```

---

## Step 2: Create Inventory

```bash id="inventory"
echo "[servers]" > inventory.ini

for i in {1..4}; do
 docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
server${i} >> inventory.ini
done

cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root

```

```
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

---

## Step 3: View Inventory

```bash id="viewinventory"
cat inventory.ini
```

---

## Step 4: Test Connectivity

```bash id="ansibleping"
ansible all -i inventory.ini -m ping
```

📸 Screenshot:
`![Verification](../screenshots/lab9/Screenshot%202026-04-30%20224523.png)`

---

## Step 5: Create Playbook

```yaml id="playbook"

- name: Update servers
 hosts: all
 become: yes

 tasks:
 - name: Update packages
 apt:
 update_cache: yes
```

---

## Step 6: Run Playbook

```bash id="runplaybook"
ansible-playbook -i inventory.ini playbook1.yml
```

📸 Screenshot:
`![Verification](../screenshots/lab9/Screenshot%202026-04-30%20224545.png)`

---

## Step 7: Verify

```bash id="verify"
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```
```

📸 Screenshot:

```
```bash id="verify"
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```
```

📸 Screenshot:

```
`![Verification](../screenshots/lab9/Screenshot%202026-04-30%20224557.png)
```

Conclusion

Ansible simplifies server management by automating tasks and ensuring consistency across multiple systems.

Key Takeaways

- * Ansible is agentless
- * Uses SSH
- * Playbooks automate tasks
- * Docker simulates servers

Experiment 10

Working with SonarQube

Introduction

What is SonarQube?

SonarQube is an open-source platform for continuous inspection of code quality. It performs static code analysis to detect:

- Bugs
 - Code smells
 - Security vulnerabilities
-

How SonarQube Solves the Problem

- **Continuous Inspection** → Scans code on every commit
 - **Quality Gates** → Defines pass/fail criteria
 - **Technical Debt** → Measures effort to fix issues
 - **Multi-language Support** → Supports 20+ languages
 - **Visual Dashboard** → Displays code metrics
-

Key Terms

| Term | Meaning |
|----------------|------------------------------------|
| Quality Gate | Criteria to pass before deployment |
| Bug | Code that may break |
| Vulnerability | Security issue |
| Code Smell | Poorly written code |
| Technical Debt | Time required to fix issues |
| Coverage | % of tested code |
| Duplication | Repeated code |

Practical

Step 1: Create `docker-compose.yml`

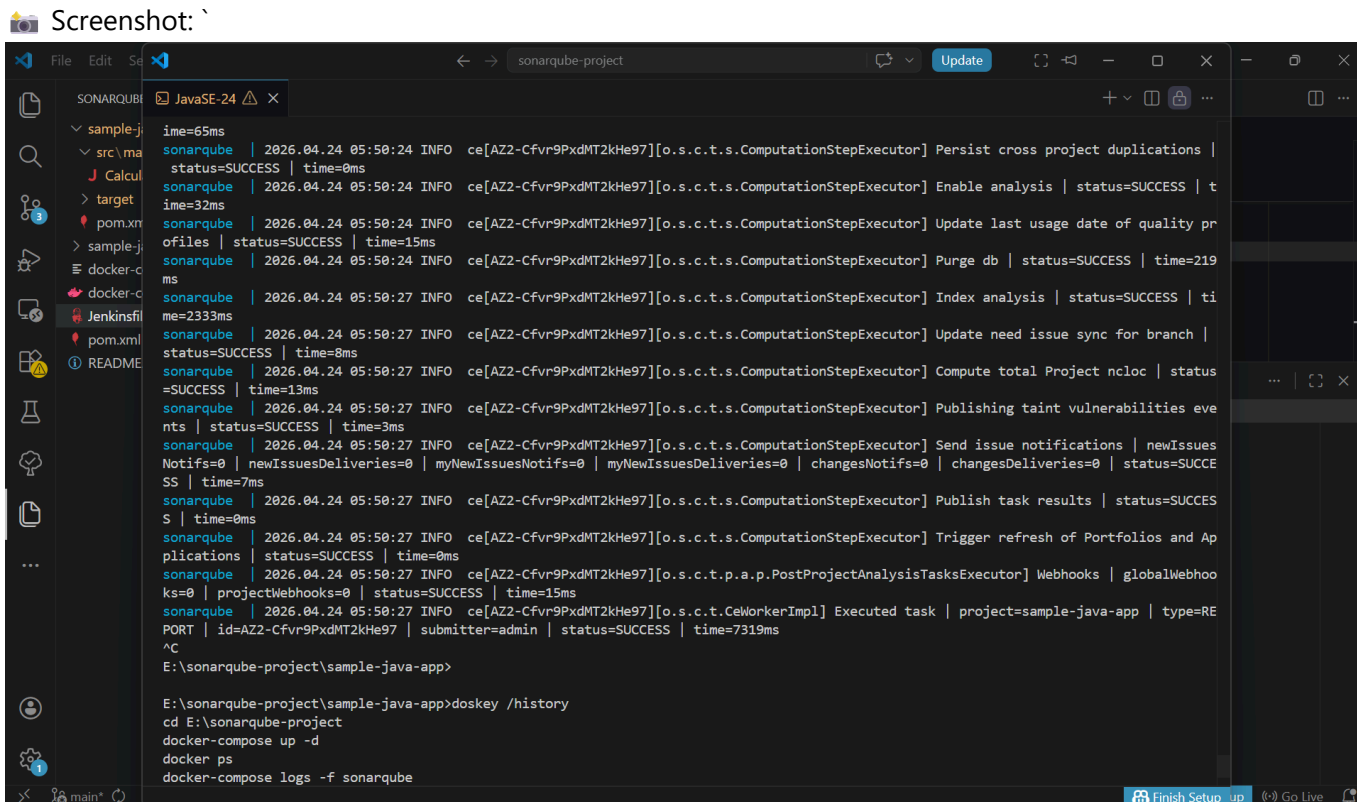
```
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

networks:
  sonarqube-lab:
    driver: bridge
```



Step 2: Start SonarQube

```
docker-compose up -d
```

Screenshot:

```
docker-compose up -d
```

```
```
```

Screenshot:

```
`![Docker Compose Up](../screenshots/lab10/Screenshot%20(710).png)
```

```

```

## Step 3: Verify Using Logs

```
```bash id="logs"
docker-compose logs -f sonarqube
```
```

```

```

## Step 4: Login to SonarQube

👉 Open in browser:  
<http://localhost:9000>



```
* Username: `admin`
* Password: `admin`
```

 Screenshot:  
`![Login](../screenshots/lab10/Screenshot%202026-04-30%20232121.png)`

---

## Step 5: Home Page

 Screenshot:  
`![Homepage](../screenshots/lab10/Screenshot%202026-04-30%20232136.png)`

---

## Step 6: Create Sample Java App

```
`bash id="mkdir"
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
`
```

## Step 7: Create `Calculator.java`

```
`java id="java"
package com.example;

public class Calculator {

 public int divide(int a, int b) {
 return a / b;
 }

 public int add(int a, int b) {
 int result = a + b;
 int unused = 100;
 return result;
 }

 public String getUser(String userId) {
 String query = "SELECT * FROM users WHERE id = " + userId;
 return query;
 }

 public int multiply(int a, int b) {
 int result = 0;
 for (int i = 0; i < b; i++) {
 result = result + a;
 }
 return result;
 }

 public int multiplyAlt(int a, int b) {
```

```

 int result = 0;
 for (int i = 0; i < b; i++) {
 result = result + a;
 }
 return result;
 }

 public String getName(String name) {
 return name.toUpperCase();
 }

 public void riskyOperation() {
 try {
 int x = 10 / 0;
 } catch (Exception e) {
 }
 }
}
...

```

🖼️ Screenshot:

^[Java File](../screenshots/lab10/Screenshot%202026-04-30%20232232.png)

---

## Step 8: Create `pom.xml`

```

<?xml id="pom"
<project xmlns="http://maven.apache.org/POM/4.0.0">
 <modelVersion>4.0.0</modelVersion>

 <groupId>com.example</groupId>
 <artifactId>sample-app</artifactId>
 <version>1.0-SNAPSHOT</version>

 <properties>
 <sonar.projectKey>sample-java-app</sonar.projectKey>
 <sonar.host.url>http://localhost:9000</sonar.host.url>
 <sonar.login>YOUR_TOKEN_HERE</sonar.login>
 </properties>

 <dependencies>
 <dependency>
 <groupId>junit</groupId>
 <artifactId>junit</artifactId>
 <version>4.13.2</version>
 <scope>test</scope>
 </dependency>
 </dependencies>

 <build>
 <plugins>
 <plugin>
 <groupId>org.sonarsource.scanner.maven</groupId>

```

```

 <artifactId>sonar-maven-plugin</artifactId>
 </plugin>
 </plugins>
 </build>
</project>
...

```

## ## Step 9: Generate Token

\* Go to: \*\*Profile → My Account → Security\*\*

\* Create token: ````xml id="pom"

```

<project xmlns="http://maven.apache.org/POM/4.0.0">
 <modelVersion>4.0.0</modelVersion>

```

```

 <groupId>com.example</groupId>
 <artifactId>sample-app</artifactId>
 <version>1.0-SNAPSHOT</version>

```

```

<properties>
 <sonar.projectKey>sample-java-app</sonar.projectKey>
 <sonar.host.url>http://localhost:9000</sonar.host.url>
 <sonar.login>YOUR_TOKEN_HERE</sonar.login>
</properties>

```

```

<dependencies>
 <dependency>
 <groupId>junit</groupId>
 <artifactId>junit</artifactId>
 <version>4.13.2</version>
 <scope>test</scope>
 </dependency>
</dependencies>

```

```

<build>
 <plugins>
 <plugin>
 <groupId>org.sonarsource.scanner.maven</groupId>
 <artifactId>sonar-maven-plugin</artifactId>
 </plugin>
 </plugins>
</build>
</project>
...

```

## ## Step 9: Generate Token

\* Go to: \*\*Profile → My Account → Security\*\*

\* Create token: `scanner-token`

---

## ## Step 10: Run Scanner

```
```bash id="scanner"
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```
```

📷 Screenshot:

```
```bash id="scanner"
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```
```

📷 Screenshot:

```
`![Scanner Run](../screenshots/lab10/Screenshot%20(707).png)
```

---

## Step 11: Verify Build Success

📷 Screenshot:

```
`![Build Success](../screenshots/lab10/Screenshot%202026-04-30%20232343.png)
```

---

## Step 12: View Dashboard

👉 Open:

<http://localhost:9000/dashboard?id=sample-java-app>

📷 Screenshot:

```
`![Dashboard](../screenshots/lab10/Screenshot%20(706).png)
```

📷 Screenshot:

```
`![Dashboard](../screenshots/lab10/Screenshot%202026-04-20%20114053.png)
```

📷 Screenshot:

```
`![Report](../screenshots/lab10/Screenshot%202026-04-20%20103310.png)
```

📷 Screenshot:

```
`![Report](../screenshots/lab10/Screenshot%202026-04-18%20151951.png)
```

---

## Step 13: View Detailed Report

- \* Click issues
- \* View exact line and reason

📷 Screenshot:

```
`![Report](../screenshots/lab10/Screenshot%202026-04-18%20151930.png)
```

---

# Conclusion

SonarQube helps in maintaining code quality by detecting bugs, vulnerabilities, and improving maintainability through continuous inspection.

---

# Key Takeaways

- \* Continuous code inspection
- \* Improves code quality
- \* Detects bugs & vulnerabilities
- \* Provides visual reports
- \* Integrates with CI/CD pipelines

---

# Experiment 11

---

## Container Orchestration with Docker Stack

---

### Objective

---

To understand and implement container orchestration using Docker Stack and Docker Swarm for deploying and managing multi-container applications across a cluster.

---

### Theory

---

#### Docker Stack

Docker Stack is a collection of services that can be deployed together using a `docker-compose.yml` file (version 3+). It enables orchestration and scaling on Docker Swarm.

---

#### Docker Swarm

Docker Swarm is Docker’s native clustering tool that turns multiple Docker hosts into a single virtual system.

#### Components

- **Manager Node** → Manages cluster and scheduling
  - **Worker Node** → Executes tasks
  - **Service** → Definition of containers
  - **Task** → Running container instance
- 

#### Key Concepts

| Concept         | Description                 |
|-----------------|-----------------------------|
| Stack           | Group of services           |
| Service         | Container definition        |
| Replica         | Number of instances         |
| Overlay Network | Communication between nodes |
| Volume          | Persistent storage          |

---

### Prerequisites

---

- Docker installed
- Docker Compose v3+
- Basic Docker knowledge
- Terminal access

## Environment Setup

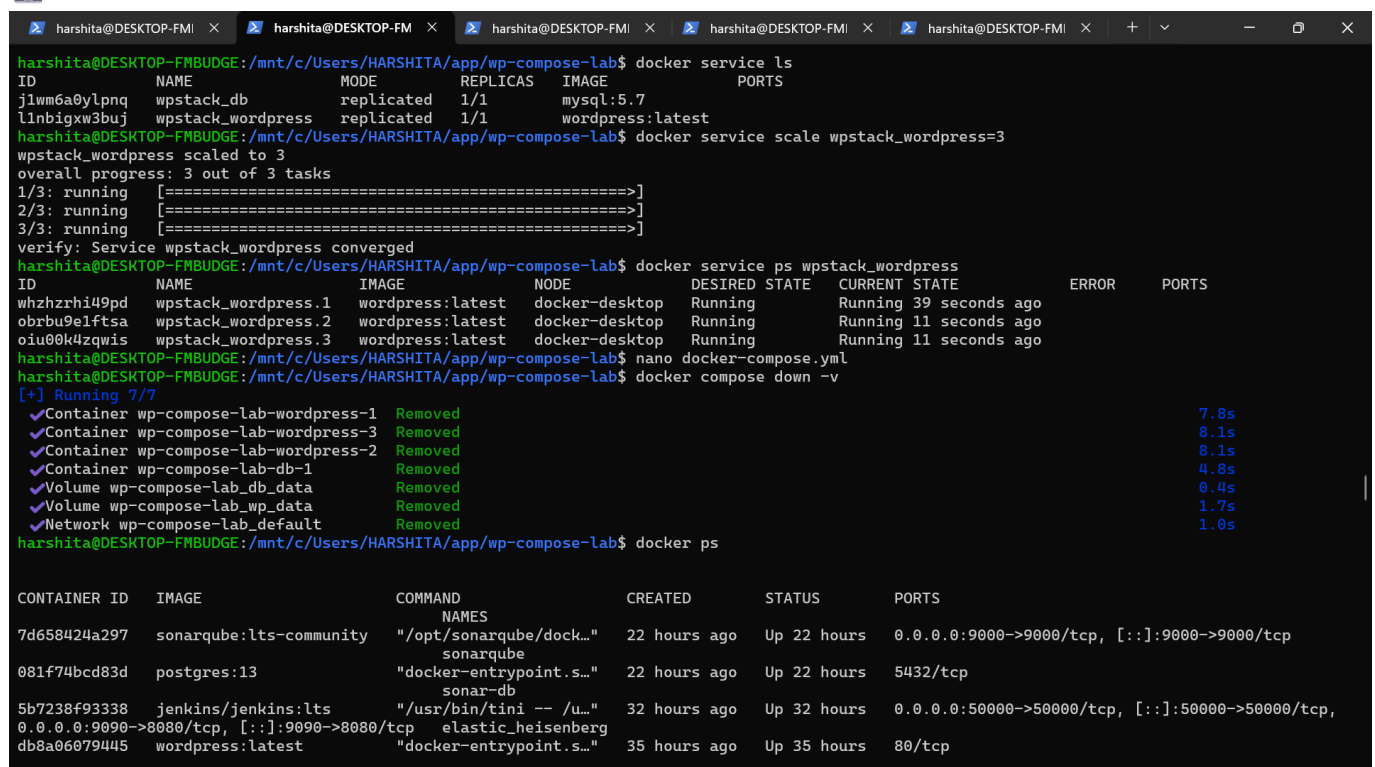
### Check Docker Version

```
docker --version
docker-compose --version
```

### Initialize Swarm

```
docker swarm init
```

 Screenshot: `



```
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
jlwm6a0ylnq wpstack_db replicated 1/1 mysql:5.7
l1nbigxw3buj wpstack_wordpress replicated 1/1 wordpress:latest
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====]
2/3: running [=====]
3/3: running [=====]
verify: Service wpstack_wordpress converged
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
whzhzrhi49pd wpstack_wordpress.1 wordpress:latest docker-desktop Running Running 39 seconds ago
obrbu9elftsa wpstack_wordpress.2 wordpress:latest docker-desktop Running Running 11 seconds ago
oiu00k4zqwis wpstack_wordpress.3 wordpress:latest docker-desktop Running Running 11 seconds ago
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker compose down -v
[+] Running 7/7
✔ Container wp-compose-lab-wordpress-1 Removed 7.8s
✔ Container wp-compose-lab-wordpress-3 Removed 8.1s
✔ Container wp-compose-lab-wordpress-2 Removed 8.1s
✔ Container wp-compose-lab-db-1 Removed 4.8s
✔ Volume wp-compose-lab_db_data Removed 0.4s
✔ Volume wp-compose-lab_wp_data Removed 1.7s
✔ Network wp-compose-lab_default Removed 1.0s
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker ps

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
7d658424a297 sonarqube:lts-community "/opt/sonarqube/dock... 22 hours ago Up 22 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
081f74bcd83d postgres:13 "docker-entrypoint.s... 22 hours ago Up 22 hours 5432/tcp
5b7238f93338 jenkins/jenkins:lts "/usr/bin/tini -- /u... 32 hours ago Up 32 hours 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp elastic_heisenberg
db8a06079445 wordpress:latest "docker-entrypoint.s... 35 hours ago Up 35 hours 80/tcp
```

```

harshita@DESKTOP-FMI x harshita@DESKTOP-FM x harshita@DESKTOP-FMI x harshita@DESKTOP-FMI x harshita@DESKTOP-FMI x + - [] x
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker swarm init
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join another one.
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker node ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
i0vedxjp8a8exw8t4homgdj5a * docker-desktop Ready Active Leader 28.4.0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service wpstack_wordpress (id: l1nbixw3bujx2snc3u56wk8t)
Updating service wpstack_db (id: j1wm6a0ylpnqeda5ucfwg7enf)
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
j1wm6a0ylpnq wpstack_db replicated 1/1 mysql:5.7
l1nbixw3buj wpstack_wordpress replicated 1/1 wordpress:latest
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
whzhzrhi49pd wpstack_wordpress.1 wordpress:latest docker-desktop Running Running 35 hours ago
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
7d658424a297 sonarqube:lts-community "/opt/sonarqube/dock... 22 hours ago Up 22 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
081f74bcd83d postgres:13 "docker-entrypoint.s..." 22 hours ago Up 22 hours 5432/tcp
5b7238f93338 jenkins/jenkins:lts "/usr/bin/tini -- /u..." 32 hours ago Up 32 hours 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp,
0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp
8ffad0f23fc4 wordpress:latest "docker-entrypoint.s..." 35 hours ago Up 35 hours 80/tcp
8efcde0775f9 mysql:5.7 "docker-entrypoint.s..." 35 hours ago Up 35 hours 3306/tcp, 33060/tcp
ba4733dfb8c0 app-nodeapp "docker-entrypoint.s..." 36 hours ago Up 36 hours 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
multi-stage-app
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

```

## Verify Swarm

```

docker info | grep -i swarm
docker node ls

```



Screenshot:

```

harshita@DESKTOP-FMI x harshita@DESKTOP-FM x harshita@DESKTOP-FMI x harshita@DESKTOP-FMI x harshita@DESKTOP-FMI x + - [] x
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker swarm init
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join another one.
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker node ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
i0vedxjp8a8exw8t4homgdj5a * docker-desktop Ready Active Leader 28.4.0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service wpstack_wordpress (id: l1nbixw3bujx2snc3u56wk8t)
Updating service wpstack_db (id: j1wm6a0ylpnqeda5ucfwg7enf)
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
j1wm6a0ylpnq wpstack_db replicated 1/1 mysql:5.7
l1nbixw3buj wpstack_wordpress replicated 1/1 wordpress:latest
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
whzhzrhi49pd wpstack_wordpress.1 wordpress:latest docker-desktop Running Running 35 hours ago
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
7d658424a297 sonarqube:lts-community "/opt/sonarqube/dock... 22 hours ago Up 22 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
081f74bcd83d postgres:13 "docker-entrypoint.s..." 22 hours ago Up 22 hours 5432/tcp
5b7238f93338 jenkins/jenkins:lts "/usr/bin/tini -- /u..." 32 hours ago Up 32 hours 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp,
0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp
8ffad0f23fc4 wordpress:latest "docker-entrypoint.s..." 35 hours ago Up 35 hours 80/tcp
8efcde0775f9 mysql:5.7 "docker-entrypoint.s..." 35 hours ago Up 35 hours 3306/tcp, 33060/tcp
ba4733dfb8c0 app-nodeapp "docker-entrypoint.s..." 36 hours ago Up 36 hours 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
multi-stage-app
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

```



# Experiment Steps

## Step 0: docker-compose.yml



Screenshot: `

```
multi-stage-app
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service wpstack_wordpress (id: l1nbixw3bujx2snc3u56wk8t)
Updating service wpstack_db (id: j1wm6a0ylpnqeda5ucfwg7enf)
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
j1wm6a0ylpnq wpstack_db replicated 1/1 mysql:5.7
l1nbixw3buj wpstack_wordpress replicated 1/1 wordpress:latest *:8080->80/tcp
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====]
2/3: running [=====]
3/3: running [=====]
verify: Service wpstack_wordpress converged
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
vv7uijyd87ng wpstack_db replicated 1/1 mysql:5.7
incco86y340f wpstack_wordpress replicated 3/3 wordpress:latest *:8081->80/tcp
```

## Step 1: Stop Existing Containers

```
docker compose down
docker ps
```

## Step 2: Initialize Swarm

```
docker swarm init
```

## Step 3: Verify Nodes

```
docker node ls
```

 Screenshot:

```
harshita@DESKTOP-FMI x harshita@DESKTOP-FM x harshita@DESKTOP-FMI x harshita@DESKTOP-FMI x harshita@DESKTOP-FMI x + - [icon] x
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker swarm init
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join another one.
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker node ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
i0vedxjp8a8exw8t4homgdj5a * docker-desktop Ready Active Leader 28.4.0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service wpstack_wordpress (id: l1nbixw3bujx2snc3u56wk8t)
Updating service wpstack_db (id: j1wm6a0ylnqeda5ucfwg7enf)
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
j1wm6a0ylnpq wpstack_db replicated 1/1 mysql:5.7
l1nbixw3buj wpstack_wordpress replicated 1/1 wordpress:latest
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
whzhzrhi49pd wpstack_wordpress.1 wordpress:latest docker-desktop Running Running 35 hours ago
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
7d658424a297 sonarqube:lts-community "/opt/sonarqube/dock... 22 hours ago Up 22 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
081f74bcd83d postgres:13 "docker-entrypoint.s..." 22 hours ago Up 22 hours 5432/tcp
5b7238f93338 jenkins/jenkins:lts "/usr/bin/tini -- /u..." 32 hours ago Up 32 hours 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp,
0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp
8ffad0f23fc4 wordpress:latest "docker-entrypoint.s..." 35 hours ago Up 35 hours 80/tcp
8efcde0775f9 mysql:5.7 "docker-entrypoint.s..." 35 hours ago Up 35 hours 3306/tcp, 33060/tcp
ba4733dfb8c0 app-nodeapp "docker-entrypoint.s..." 36 hours ago Up 36 hours 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
multi-stage-app
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ nano docker-compose.yml
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA/app/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart
```

## Step 4: Deploy Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

 Screenshot:

```
docker stack deploy -c docker-compose.yml wpstack
```
```

 Screenshot:

```
`![Stack Deploy](../screenshots/lab11/Screenshot%20(750).png)
```

Step 5: List Services

```
```bash id="services"
docker service ls
```
```

 Screenshot:

```
`![Service List](../screenshots/lab11/Screenshot%20(750).png)
```

Step 6: Inspect Service Tasks

```
```bash id="tasks"
docker service ps wpstack_wordpress
```
```

📷 Screenshot:

```
```bash id="tasks"
docker service ps wpstack_wordpress
```
```

📷 Screenshot:

```
`![Service Tasks](../screenshots/lab11/Screenshot%20(751).png)
```

Step 7: Check Running Containers

```
```bash id="containers"
docker ps
```
```

Step 8: Scale Service

```
```bash id="scale"
docker service scale wpstack_wordpress=3
```
```

Step 9: Filter WordPress Containers

```
```bash id="filter"
docker ps | grep wordpress
```
```

📷 Screenshot:

```
`![Filtered Containers](../screenshots/lab11/Screenshot%20(751).png)
```

Step 10: Remove Stack

```
```bash id="remove"
docker stack rm wpstack
```
```

📷 Screenshot:

```
```bash id="remove"
docker stack rm wpstack
```
```

📷 Screenshot:

```
`![Stack Remove](../screenshots/lab11/Screenshot%20(752).png)
```

Observations

| Parameter | Observed Value |
|------------------------|----------------|
| ----- | ----- |
| Stack Name | wpstack |
| Services Deployed | ___ |
| Initial Replicas | ___ |
| Replicas After Scaling | 3 |
| WordPress Port | ___ |
| MySQL Service Name | ___ |
| Overlay Network | ___ |

Expected Output

- * ``docker stack ls`` → shows stack
- * ``docker service ls`` → shows services
- * WordPress accessible on browser

Docker Stack vs Docker Compose

| Feature | Docker Compose | Docker Stack |
|----------------|----------------|--------------|
| ----- | ----- | ----- |
| Scope | Single host | Multi-node |
| Swarm Required | No | Yes |
| Scaling | Manual | Declarative |
| Load Balancing | No | Yes |
| Updates | No | Rolling |

Common Commands

```
```bash id="commands"
docker stack deploy -c <file> <name>
docker stack ls
docker stack services <name>
docker stack ps <name>
docker stack rm <name>
docker service scale <service>=<replicas>
docker service logs <service>
```
```

Troubleshooting

| Problem | Cause | Solution |
|---------|-------|----------|
| ----- | ----- | ----- |

| | | | |
|-----------------------|--------------------------|-------------|--|
| Service not starting | Image missing | Build image | |
| Swarm not initialized | Not in swarm mode | Run init | |
| Port conflict | Port busy | Change port | |
| Pending replicas | Resource issue | Check nodes | |

Viva Questions

- * Difference between Docker Compose and Stack?
- * What is Docker Swarm?
- * What is overlay network?
- * How scaling works?
- * What is service vs task?

Conclusion

- * Swarm cluster initialized
- * Stack deployed successfully
- * Services scaled
- * Monitoring performed

Docker Stack provides production-ready orchestration with scaling, load balancing, and fault tolerance.

References

- * Docker Stack Docs
- * Docker Swarm Docs
- * Compose v3 Docs

Experiment 12

Container Orchestration using Kubernetes

Objective

To learn Kubernetes basics by deploying a WordPress application using Minikube, including:

- Deployments
 - Services
 - Scaling Pods
 - Self-healing
-

Tools & Environment

- Minikube
 - kubectl
 - Ubuntu 22.04
 - Docker
-

Theory

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.

Key Concepts

| Term | Description |
|------------|------------------------------------|
| Pod | Smallest unit (contains container) |
| Deployment | Manages replicas of pods |
| Service | Exposes pods |
| NodePort | External access to service |
| ReplicaSet | Maintains pod count |
| kubectl | CLI tool |

Steps Performed

Step 1: Create Deployment YAML

```
nano wordpress-deployment.yaml
```

wordpress-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress

spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress

  template:
    metadata:
      labels:
        app: wordpress

    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```

 Screenshot:

```

harshita@DESKTOP-FMI x  harshita@DESKTOP-FMI x  harshita@DESKTOP-FMI x  harshita@DESKTOP-FMI x  harshita@DESKTOP-FM x  +  -  [icon]  X

This message is shown once a day. To disable it please create the
/home/harshita/.hushlogin file.
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ minikube version
Command 'minikube' not found, did you mean:
  command 'minitube' from deb minitube (3.9.3-2)
Try: sudo apt install <deb name>
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ k3d version
Command 'k3d' not found, did you mean:
  command 'k3b' from snap k3b (26.04.0)
  command 'f3d' from deb f3d (2.2.1+dfsg-2)
  command 'k3b' from deb k3b (23.08.4-0ubuntu1)
  command 'kxd' from deb kxd (0.15-4.1)
  command 's3d' from deb s3d (0.2.2.1-6build1)
See 'snap info <snapname>' for additional versions.
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ kubectl version --client
Client Version: v1.32.2
Kustomize Version: v5.5.0
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
  % Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
100 128M 100 128M    0     0  9.9M      0  0:00:12  0:00:12 --:--:-- 11.2M
[sudo] password for harshita:
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ minikube start
🐳 minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)

^C
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
Preparing to install k3d into /usr/local/bin
k3d installed into /usr/local/bin/k3d
Run 'k3d --help' to see what you can do with it.
harshita@DESKTOP-FMBUDGE:/mnt/c/Users/HARSHITA$ k3d cluster create mycluster
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-mycluster'
INFO[0001] Created image volume k3d-mycluster-images
INFO[0001] Starting new tools node...
INFO[0002] Creating node 'k3d-mycluster-server-0'

```

Step 2: Start Minikube & Apply Deployment

```
minikube start
kubectl apply -f wordpress-deployment.yaml
```

 Screenshot:

```
minikube start
kubectl apply -f wordpress-deployment.yaml
^^^
```

 Screenshot:

```
^[Minikube Start](../screenshots/lab12/Screenshot%20(725).png)
```

Step 3: Create Service YAML

```
```bash id="nano2"
nano wordpress-service.yaml
```
```

```
### wordpress-service.yaml
```

```
```yaml id="service"
apiVersion: v1
kind: Service
```



```
metadata:
 name: wordpress-service

spec:
 type: NodePort
 selector:
 app: wordpress
```

```
 ports:
 - port: 80
 targetPort: 80
 nodePort: 30007

```

## Step 4: Apply Service

```
```bash id="appliesvc"
kubectl apply -f wordpress-service.yaml
```
```

📸 Screenshot:

```
`![Service Apply](../screenshots/lab12/Screenshot%20(726).png)
```

## Step 5: Verify Pods

```
```bash id="pods"
kubectl get pods
```
```

📸 Screenshot:

```
```bash id="pods"
kubectl get pods
```
```

📸 Screenshot:

```
`![Pods](../screenshots/lab12/Screenshot%20(726).png)
```

## Step 6: Verify Service

```
```bash id="svc"
kubectl get svc
```
```

## Step 7: Scale Deployment

```
```bash id="scale"
kubectl scale deployment wordpress --replicas=4
```
```

📸 Screenshot:

`![Scaling](../screenshots/lab12/Screenshot%20(726).png)`

---

## Step 8: Verify Scaled Pods

```
```bash id="pods2"
kubectl get pods
```
```

📸 Screenshot:

```
```bash id="pods2"
kubectl get pods
```
```

📸 Screenshot:

`![Scaled Pods](../screenshots/lab12/Screenshot%20(726).png)`

---

## Step 9: Delete a Pod (Self-Healing Test)

```
```bash id="delete"
kubectl delete pod <pod-name>
```
```

---

## Step 10: Verify Self-Healing

```
```bash id="verify"
kubectl get pods
```
```

📸 Screenshot:

`![Self Healing](../screenshots/lab12/Screenshot%20(726).png)`

---

# Commands Summary

| Command                    | Purpose       |
|----------------------------|---------------|
| -----                      | -----         |
| minikube start             | Start cluster |
| kubectl apply -f file.yaml | Apply config  |
| kubectl get pods           | List pods     |
| kubectl get svc            | List services |
| kubectl scale deployment   | Scale pods    |
| kubectl delete pod         | Delete pod    |

---

#### # Observations

- \* Deployment created 2 pods initially
- \* Service exposed app on NodePort 30007
- \* Scaling created additional pods instantly
- \* Self-healing replaced deleted pod automatically

---

#### # Conclusion

- \* Kubernetes manages deployments efficiently
- \* Scaling is automatic and fast
- \* Self-healing ensures availability
- \* Infrastructure complexity is abstracted

---